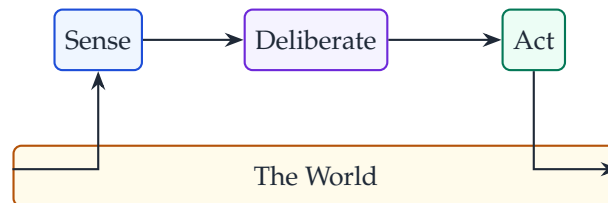

Artificial Intelligence: Search Methods for Problem Solving

Week 1 — Introduction

Detailed study notes built on the lectures of Prof. Deepak Khemani (IIT Madras)

IITM BS in Programming & Data Science



How to use these notes: each module follows the order of the slides. Yellow boxes give intuition, blue boxes are definitions, purple boxes are formulas and derivations, green boxes are side notes, red boxes flag common exam traps. The practice section at the end (numericals, subjective questions, MCQs) covers every module.

Contents

1	Course Overview and Intelligent Agents	4
1.1	Where this course sits	4
1.2	Syllabus map	4
1.3	Textbook and references	4
1.4	Intelligent agents	5
1.5	The information-processing view of AI	5
1.6	The building blocks of AI	6
1.7	What is intelligence? Past, present, future	7
2	A Decade of Machine Learning	9
2.1	Why the lecture detours into ML	9
2.2	Neurons and the perceptron	9
2.3	The hidden layer and backpropagation	11
2.4	Deep neural networks as classifiers	13
2.5	From signals to symbols: what perception delivers	14
2.6	Animal-like abilities vs human-level intelligence	15
2.7	Performance vs competence	15
2.8	Suitcase words	16
2.9	AlphaGo (2016): reinforcement learning in games	16
2.10	Human cognitive architecture	17
3	AI and ML: Knowledge, Representation and Reasoning	18
3.1	Two different jobs	18
3.2	Knowledge and reasoning: the core of intelligence	18
3.3	Representation and semiotics	19
3.4	Reasoning	20
3.5	AI = Automation?	21
4	History and Philosophy: Can Machines Think?	22
4.1	Four definitions of AI	22
4.2	Machines with minds of their own	23
4.3	The fundamental questions	23
4.4	The raging debates over thinking machines	24
4.5	Alan Turing's imitation game	24
4.6	The Loebner Prize and chatbots	25
4.7	ELIZA (1966)	26
4.8	Conversations with machines: the problem	26

4.9	Winograd schemas: an alternative Turing test	26
5	Minds and Machines Made of Matter	30
5.1	Galileo: perception is an internal process	30
5.2	Thomas Hobbes: the grandfather of AI	31
5.3	René Descartes: thoughts are symbols	31
5.4	Dualism: mind and body	32
5.5	The paradox of mechanical reason	32
6	Artificial People and Real Mechanisms	34
6.1	Artificial people in Western mythology and fiction	34
6.2	Real mechanisms: clocks, automata and thinking machines	35
6.3	Vaucanson's Duck (1739)	36
6.4	Von Kempelen's Chess-Playing Turk (1770)	36
6.5	Mechanical arithmetic	36
6.6	Leibniz's <i>calculus ratiocinator</i>	37
6.7	Commercial success: the Arithmometer	38
6.8	Charles Babbage: the first computer	38
6.9	Ada Lovelace: the first programmer	39
6.10	ENIAC: the first electronic computer	40
7	Modern Times: The Birth of AI	42
7.1	The cast of characters so far	42
7.2	How AI got its name: Dartmouth, 1956	42
7.3	Simon and Newell	44
7.4	The information-processing view: GPS and means–ends analysis	44
7.5	Physical symbol systems	45
8	Reality, Complexity and the Domains of Reasoning	47
8.1	What we see is not what really is: Copernicus	47
8.2	Reality	47
8.3	Complexity	47
8.4	The world in our minds	48
8.5	Powers of ten	48
8.6	The domains for reasoning	50
9	Problem Solving: Where Search Fits	52
9.1	A striker on a football field	52
9.2	Problem solving defined	52
9.3	The real world is complex	53
9.4	Learn to walk before you run: simplifying assumptions	53

9.5	Two ways to solve problems	54
9.6	An architecture for problem solving	55
9.7	A map colouring problem	56
Practice: Test Yourself		59
	Part A — Numerical problems	59
	Part B — Subjective questions	64
	Part C — MCQs	68

Module 1 Course Overview and Intelligent Agents

1.1 Where this course sits

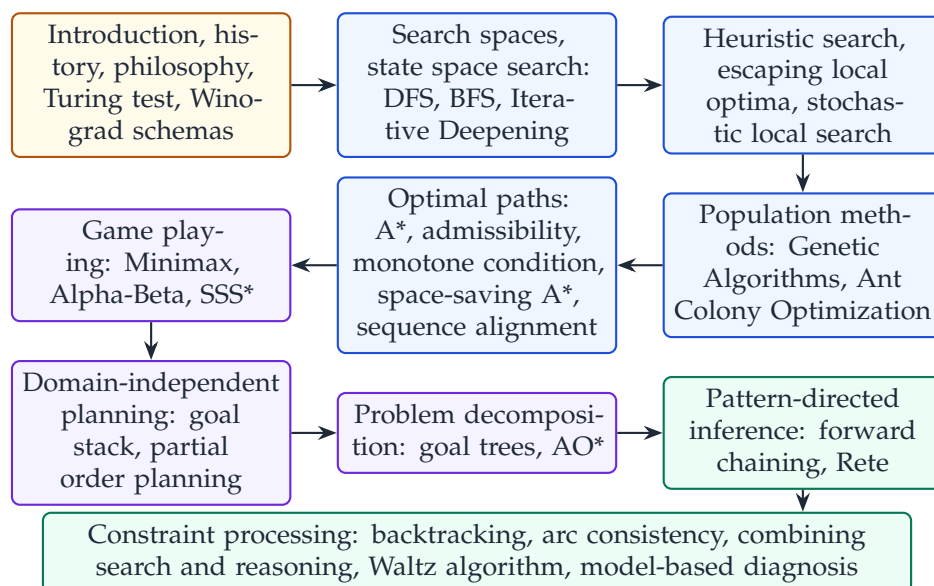
The course is titled *Search Methods for Problem Solving*. The central idea you should carry through every week is simple:

Intuition: The one-line summary of the course

When an agent does not already *know* the answer to a problem, it can still find one by **systematically exploring the space of possibilities**. Search is the “first principles” method of problem solving: slow in the worst case, but general. Everything in the syllabus is either a way to search, a way to search *smarter* (heuristics, pruning, optimality), or a way to combine search with knowledge and reasoning.

1.2 Syllabus map

The syllabus lists eleven topic clusters. It helps to see them as one story rather than a list.



Note: Reading the map

- **Yellow:** this week. Why AI, what intelligence is, where search fits.
- **Blue:** the core of search. Blind (uninformed) search first, then informed (heuristic) search, then randomised and population-based optimisation, then *optimal* path finding with A*.
- **Purple:** search in richer settings: an adversary (games), actions with preconditions and effects (planning), problems that split into sub-problems (AND-OR trees, AO*).
- **Green:** the bridge from search to *reasoning*: rule-based inference and constraint satisfaction, where search and deduction are combined.

1.3 Textbook and references

The textbook is Deepak Khemani, *A First Course in Artificial Intelligence* (McGraw Hill India, 2013). The reference list mixes three kinds of books, which is worth knowing when you want to go deeper:

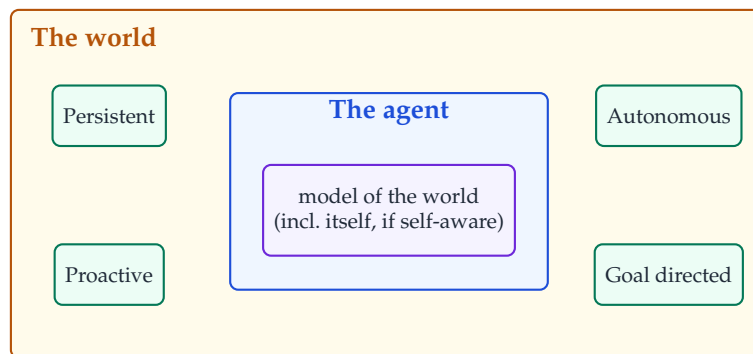
- **Search algorithms in depth:** Edelkamp & Schroedl (*Heuristic Search*), Pearl (*Heuristics*), Michalewicz & Fogel (*How to Solve It: Modern Heuristics*).
- **General AI textbooks:** Russell & Norvig (*AIMA*), Rich & Knight, Winston, Charniak & McDermott.
- **History and philosophy:** Haugeland (*AI: The Very Idea*), McCorduck (*Machines Who Think*). Most of the history in this week's slides comes from these two.

1.4 Intelligent agents

Definition: Intelligent agent

An intelligent agent is an entity that lives in a world and carries a **model of that world** inside its “head”. The model may be an *abstraction* (it keeps only what matters). A **self-aware** agent also includes *itself* in the world model.

The slide lists four properties of an agent:



- **Persistent:** it continues to exist over time; it is not a one-shot function call. It has a history and can use it.
- **Autonomous:** it makes its own decisions without a human choosing each action.
- **Proactive:** it does not just *react* to stimuli; it takes initiative to bring about states it wants.
- **Goal directed:** its behaviour is organised around goals. Goals are what make “good” and “bad” actions meaningful.

Intuition: Why a model in the head matters

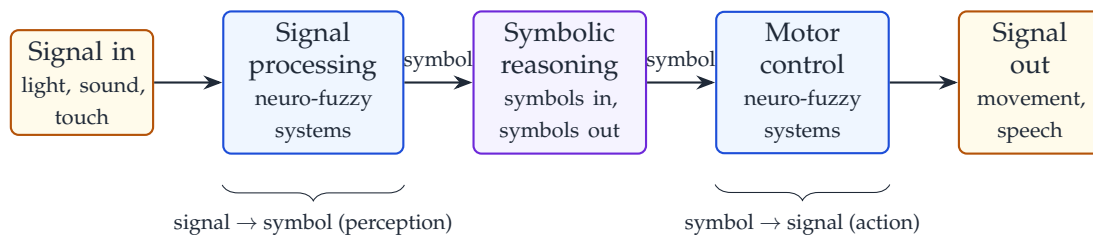
A thermostat reacts to temperature; it has no model. A chess player imagines “if I move here, she moves there” — that imagining is only possible because the player has an internal model of the board and the rules. **Search is exactly this: simulating actions inside the model before committing to them in the real world.** No model, no search.

Note: Connection to your work

An LLM “agent” in a tool-calling loop is persistent (within a session) and goal-directed, but its “world model” is implicit in weights and context. Classical AI, the subject of this course, makes the model *explicit* and *symbolic*, so that algorithms can search through it.

1.5 The information-processing view of AI

The slide shows a pipeline: **signal** → **symbol** → **signal**.



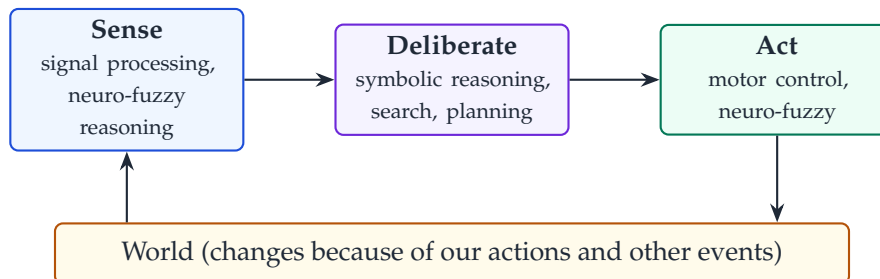
- At the edges, the agent deals with **continuous, noisy signals**. This is where “neuro-fuzzy” methods (neural networks, fuzzy logic) are strong.
- In the middle, the agent deals with **discrete symbols** (“the ball”, “defender on my left”, “goal is 20m ahead”). This is where **reasoning and search** happen.

Intuition: Why split it this way?

Your eyes receive millions of photon hits a second; you do not reason about photons. You reason about *objects*. Perception compresses a flood of signal into a few meaningful symbols; reasoning works on those symbols; action expands the chosen symbolic decision back into muscle signals. The course lives in the middle box.

1.5.1 An autonomous agent: Sense–Deliberate–Act

The next slide re-labels the same pipeline as the classic **sense–deliberate–act** cycle, and it runs in a loop because acting changes the world, which changes what you sense next.

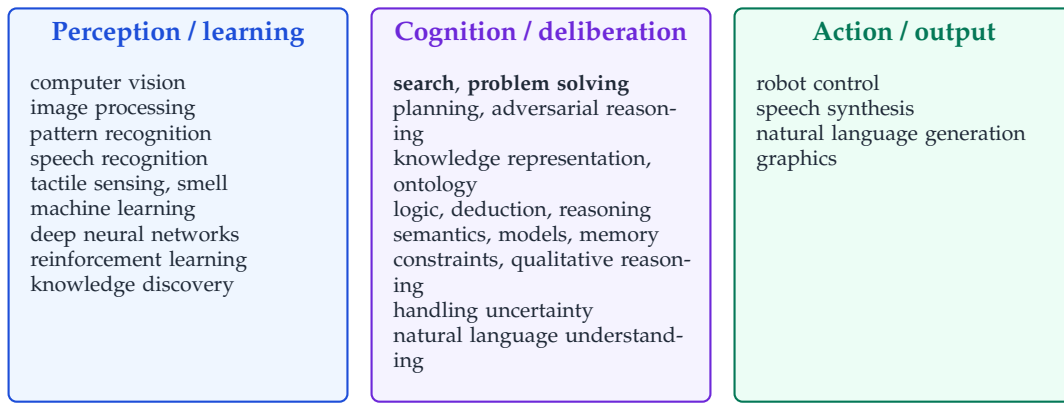


Exam trap

Do not confuse “autonomous” with “intelligent”. A washing machine runs autonomously through a program. An autonomous *intelligent* agent *deliberates*: it considers alternatives against goals using a model. The “deliberate” box is what this course is about.

1.6 The building blocks of AI

The slide scatters around thirty topic names. The useful exercise is to *sort them* by where they sit in the sense–deliberate–act pipeline.

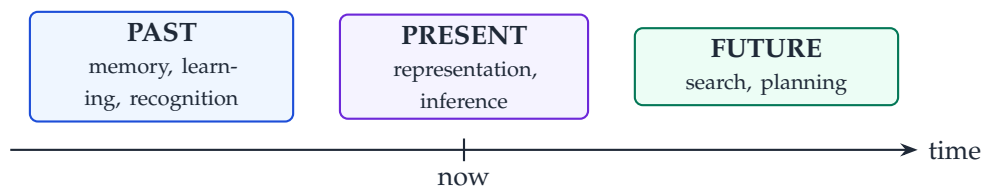
**Note**

The sorting is approximate: natural language understanding needs both perception and deep reasoning; reinforcement learning learns *how to act*. The point is that “AI” is not one technique but a whole toolkit, and this course focuses on the middle column.

1.7 What is intelligence? Past, present, future

The slide gives a neat three-part description of intelligence, organised by time:

Time	Capability	AI technique
Past	Remember the past and learn from it	Memory & experience → case-based reasoning; learning a model → machine learning; recognising objects, faces, patterns → deep neural networks
Present	Understand the present, be aware of the world	Create a model of the world → knowledge representation; make inferences from what you know → logic and reasoning
Future	Imagine the future, work towards goals	Trial and error → heuristic search ; goals, plans, actions → automated planning

**Intuition: This course = the future column**

Machine learning mostly looks *backwards* (extract patterns from data already seen). Search and planning look *forwards*: “what happens if I do this, then that?” That forward-looking imagination is the part of intelligence this course formalises.

Module recap

- An agent is persistent, autonomous, proactive and goal-directed, and keeps a (possibly abstract) model of the world; a self-aware agent models itself too.
- Information processing view: signal → symbol → signal. Perception and motor control handle signals; reasoning handles symbols.

- Sense–deliberate–act is a loop through the world. Search lives in *deliberate*.
- Intelligence spans past (learning), present (representation and inference), future (search and planning).

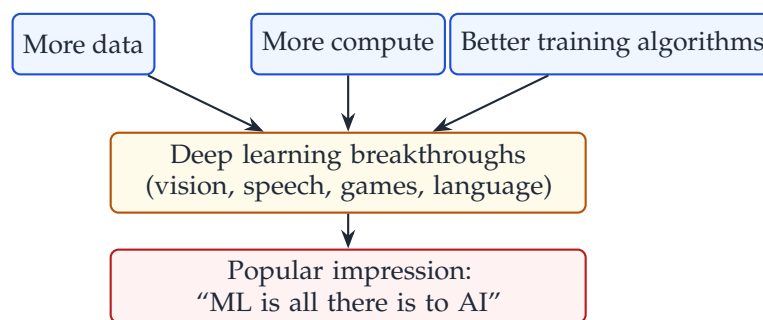
Module 2 A Decade of Machine Learning

2.1 Why the lecture detours into ML

The slides note that the last decade saw three simultaneous explosions:

1. the **amount of data** available,
2. the **computing power** available (GPUs in particular), and
3. **better algorithms for training neural networks**.

Together these created the popular impression that machine learning *is* AI. The lecture briefly examines this phenomenon, both to credit what it achieved and to show what it leaves out, before turning to search.



2.2 Neurons and the perceptron

Definition: Artificial neuron

A neuron is a *simple device that computes a simple function of its inputs*. Collections of interconnected neurons can carry out complex computations.

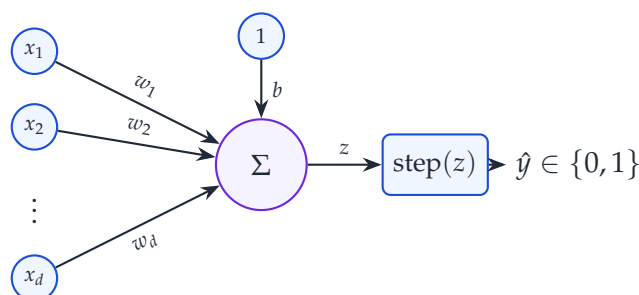
Formula / Derivation: A single neuron (perceptron unit)

Inputs $\mathbf{x} \in \mathbb{R}^d$, weights $\mathbf{w} \in \mathbb{R}^d$, bias $b \in \mathbb{R}$:

$$z = \mathbf{w}^\top \mathbf{x} + b = \sum_{i=1}^d w_i x_i + b \quad [(1 \times d)(d \times 1) + 1 \rightarrow \text{scalar}]$$

$$\hat{y} = \text{step}(z) = \begin{cases} 1 & z \geq 0 \\ 0 & z < 0 \end{cases} \quad [\text{scalar} \in \{0, 1\}]$$

Equivalently, with threshold $\theta = -b$: output 1 iff $\sum_i w_i x_i \geq \theta$.



Intuition: What a perceptron “sees”

The equation $\mathbf{w}^\top \mathbf{x} + b = 0$ is a **hyperplane** (a line in 2D, a plane in 3D). The perceptron says “class 1” on one side and “class 0” on the other. So a single perceptron can only ever draw *one straight cut* through the input space.

2.2.1 Historical claims on the slide (and the accurate record)

The slide says the perceptron was invented by McCulloch and Pitts in 1943 as a single-layer binary classifier, and that Minsky and Papert showed in 1958 that it could only classify linearly separable classes.

Exam trap: Know both versions

The standard historical record is slightly different:

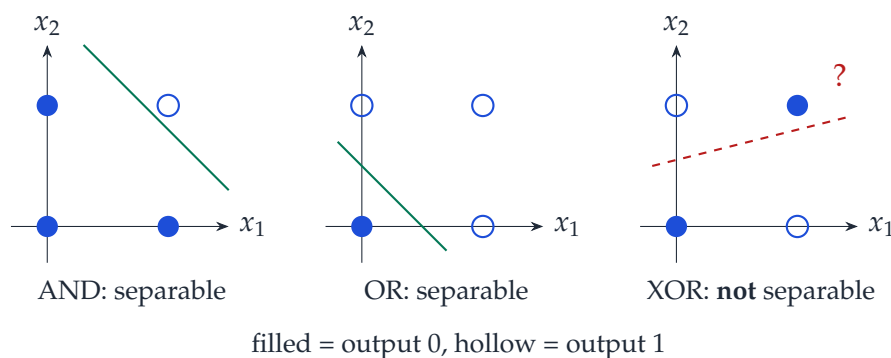
- **1943:** McCulloch & Pitts proposed the *threshold neuron* model (a logical unit, fixed weights).
- **1958:** Frank Rosenblatt introduced the *perceptron* with a learning rule.
- **1969:** Minsky & Papert’s book *Perceptrons* proved the linear-separability limitation.

If a graded question quotes the slide verbatim, expect the slide’s framing; but the substantive point both versions agree on is: **a single-layer perceptron can only separate linearly separable classes.**

2.2.2 Linear separability**Definition: Linearly separable**

Two finite sets $A, B \subset \mathbb{R}^d$ are linearly separable if there exist $\mathbf{w} \in \mathbb{R}^d, b \in \mathbb{R}$ with $\mathbf{w}^\top \mathbf{x} + b > 0$ for all $\mathbf{x} \in A$ and $\mathbf{w}^\top \mathbf{x} + b < 0$ for all $\mathbf{x} \in B$.

The slide’s picture: shaded circles (class A) and empty circles (class B) arranged so that no single line separates them. The classic minimal example is XOR:

**Formula / Derivation: Proof that no perceptron computes XOR**

Suppose weights w_1, w_2 and threshold θ realise XOR (output 1 iff $w_1x_1 + w_2x_2 \geq \theta$). Then the four inputs require:

$$(0, 0) \mapsto 0 : 0 < \theta \quad (1)$$

$$(1, 0) \mapsto 1 : w_1 \geq \theta \quad (2)$$

$$(0, 1) \mapsto 1 : w_2 \geq \theta \quad (3)$$

$$(1, 1) \mapsto 0 : w_1 + w_2 < \theta \quad (4)$$

Adding (2) and (3): $w_1 + w_2 \geq 2\theta$. By (1), $\theta > 0$ so $2\theta > \theta$, giving $w_1 + w_2 > \theta$. This contradicts (4). ■

2.2.3 The perceptron learning rule

Although the slides do not derive it, you should know how a perceptron *learns*, since “learning” is the word that drives the whole ML discussion.

Formula / Derivation: Perceptron update

For a training example $(\mathbf{x}, y)$ with prediction $\hat{y}$ and learning rate $\eta > 0$:

$$\mathbf{w} \leftarrow \mathbf{w} + \eta (y - \hat{y}) \mathbf{x} \quad [d \times 1], \quad b \leftarrow b + \eta (y - \hat{y}) \quad [\text{scalar}]$$

Why it works: if $y = 1$ but $\hat{y} = 0$, z was too small; adding $\eta \mathbf{x}$ to $\mathbf{w}$ increases z on this input by $\eta \|\mathbf{x}\|^2 \geq 0$. The opposite case decreases z . Correct predictions cause no change.

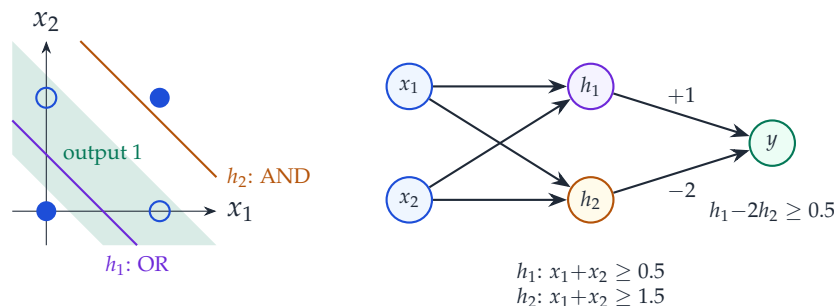
Convergence theorem: if the data is linearly separable, this rule reaches a separating hyperplane in finitely many updates. If it is *not* separable (XOR), it cycles forever.

2.3 The hidden layer and backpropagation

According to the slide, in 1986 Rumelhart, Hinton and Williams showed that a **multi-layer perceptron** (MLP) can be trained to learn non-linear classifiers, and popularised the **backpropagation** algorithm, which sends the error at the output back through the hidden layer to adjust connection weights.

Intuition: Why a hidden layer fixes XOR

Each hidden unit draws one line. The output unit then combines the half-planes. XOR = “OR but not AND”: one hidden unit computes OR, another computes AND, and the output fires when OR is on and AND is off. Two lines carve out the diagonal band that one line could not.



2.3.1 MLP forward pass with dimensions

Formula / Derivation: Two-layer network

Input $\mathbf{x} \in \mathbb{R}^d$, hidden width h , K output classes (e.g. Horse, Cow, Giraffe, Bear, Lion gives

$K = 5$).

$$\mathbf{z}^{(1)} = W^{(1)}\mathbf{x} + \mathbf{b}^{(1)}$$

$[(h \times d)(d \times 1) + (h \times 1) = h \times 1]$

$$\mathbf{a}^{(1)} = \sigma(\mathbf{z}^{(1)})$$

$[h \times 1, \text{elementwise}]$

$$\mathbf{z}^{(2)} = W^{(2)}\mathbf{a}^{(1)} + \mathbf{b}^{(2)}$$

$[(K \times h)(h \times 1) + (K \times 1) = K \times 1]$

$$\hat{\mathbf{y}} = \text{softmax}(\mathbf{z}^{(2)}), \quad \hat{y}_k = \frac{e^{z_k^{(2)}}}{\sum_{j=1}^K e^{z_j^{(2)}}}$$

$[K \times 1, \sum_k \hat{y}_k = 1]$

Parameter count: $hd + h + Kh + K$.

The step function has zero derivative almost everywhere, so gradient-based training needs a smooth activation. The classical choice was the sigmoid:

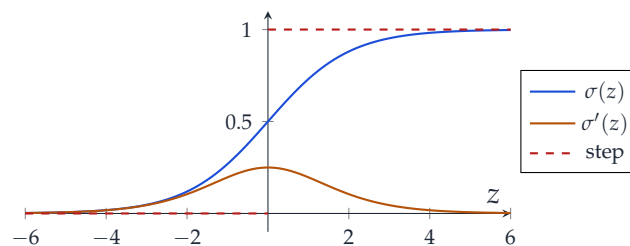
Formula / Derivation: Sigmoid and its derivative

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Derivative:

$$\begin{aligned} \sigma'(z) &= \frac{d}{dz} (1 + e^{-z})^{-1} = -(1 + e^{-z})^{-2} \cdot (-e^{-z}) = \frac{e^{-z}}{(1 + e^{-z})^2} \\ &= \underbrace{\frac{1}{1 + e^{-z}}}_{\sigma(z)} \cdot \underbrace{\frac{e^{-z}}{1 + e^{-z}}}_{1 - \sigma(z)} = \sigma(z)(1 - \sigma(z)). \end{aligned}$$

Maximum value $\sigma'(0) = 0.25$. This small maximum is one reason deep sigmoid networks suffered from *vanishing gradients*, later fixed by ReLU.



2.3.2 Backpropagation derivation

Formula / Derivation: Backprop for the two-layer network with softmax + cross-entropy

Loss for one example with one-hot target $\mathbf{y} \in \{0, 1\}^K$:

$$L = - \sum_{k=1}^K y_k \log \hat{y}_k \quad [\text{scalar}]$$

Step 1 (output layer). A standard result for softmax + cross-entropy:

$$\delta^{(2)} \equiv \frac{\partial L}{\partial \mathbf{z}^{(2)}} = \hat{\mathbf{y}} - \mathbf{y} \quad [K \times 1]$$

Derivation: $\partial \hat{y}_j / \partial z_k = \hat{y}_j (\mathbb{1}[j=k] - \hat{y}_k)$, so $\partial L / \partial z_k = - \sum_j y_j \frac{\partial \log \hat{y}_j}{\partial z_k} = - \sum_j y_j \frac{1}{\hat{y}_j} (\hat{y}_j (\mathbb{1}[j=k] - \hat{y}_k)) = -y_k + \hat{y}_k \sum_j y_j = \hat{y}_k - y_k$ (since $\sum_j y_j = 1$).

Step 2 (output weights). Since $z_k^{(2)} = \sum_i W_{ki}^{(2)} a_i^{(1)} + b_k^{(2)}$:

$$\frac{\partial L}{\partial W^{(2)}} = \delta^{(2)} \mathbf{a}^{(1)\top} \quad [(K \times 1)(1 \times h) = K \times h], \quad \frac{\partial L}{\partial \mathbf{b}^{(2)}} = \delta^{(2)} \quad [K \times 1]$$

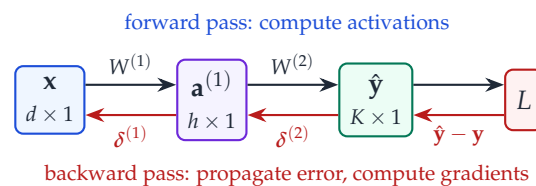
Step 3 (send error back through the hidden layer). Chain rule through $\mathbf{a}^{(1)} = \sigma(\mathbf{z}^{(1)})$:

$$\delta^{(1)} = (W^{(2)\top} \delta^{(2)}) \odot \sigma'(\mathbf{z}^{(1)}) \quad [(h \times K)(K \times 1) \odot (h \times 1) = h \times 1]$$

Step 4 (hidden weights).

$$\frac{\partial L}{\partial W^{(1)}} = \delta^{(1)} \mathbf{x}^\top \quad [(h \times 1)(1 \times d) = h \times d], \quad \frac{\partial L}{\partial \mathbf{b}^{(1)}} = \delta^{(1)}$$

Step 5 (gradient descent). $W \leftarrow W - \eta \partial L / \partial W$ for every weight matrix.



Intuition: Backprop is credit assignment

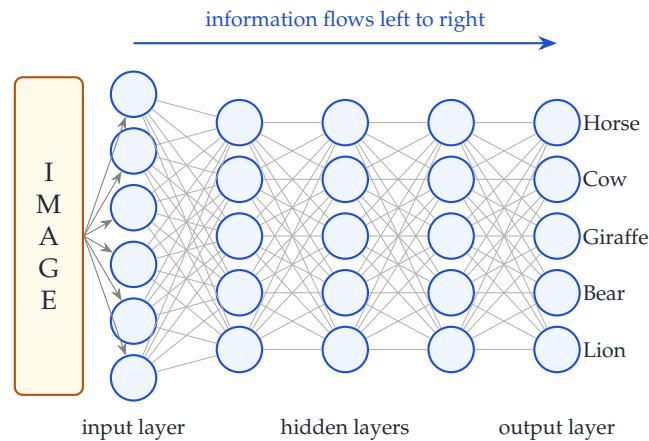
When the network gets an answer wrong, every weight is partly to blame. Backprop answers “how much would the loss change if I nudged *this* weight?” by reusing the error signal computed for the layer above. That reuse makes it efficient: one backward pass costs about the same as one forward pass.

2.3.3 Hinton’s persistence and 2012

The slide notes that Hinton persevered with neural networks and showed in 2012 that deep networks with many hidden layers achieved remarkable success in computer vision, recognising thousands of object types. (This is the AlexNet result on ImageNet, by Krizhevsky, Sutskever and Hinton.) Hinton, LeCun and Bengio received the **2018 Turing Award**.

2.4 Deep neural networks as classifiers

Three slides show the same architecture with different output labels: animals (Horse, Cow, Giraffe, Bear, Lion), scene elements (Tree, River, Sky, Hill, Grass), fruits (Pear, Litchi, Plum, Guava, Orange). Information flows left to right: this is a **feed-forward** network.



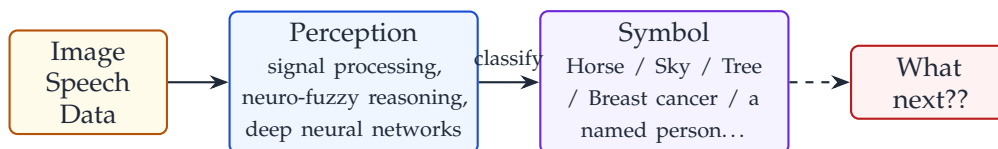
Note: Same architecture, different labels

The three slides make a quiet but important point: **the network's structure does not know what the labels mean**. Swap "Horse" for "Pear" and retrain, and the same machinery works. The labels are just output positions; their meaning lives in the humans who chose them. This sets up the discussion of symbols below.

2.5 From signals to symbols: what perception delivers

Definition: Symbol (as used in the slides)

A symbol is a perceptible something that **stands for something else**. It has **no intrinsic meaning**; its meaning is assigned.



Intuition: The "What next??" slide

Deep learning excels at turning raw signals into labels: this is a horse, this is sky, this scan shows cancer, this face is a particular person. But a label is where the interesting work *starts*, not where it ends. Once you know "defender approaching from the left", you still have to *decide what to do*. That decision process, reasoning over symbols towards a goal, is what classification does not supply.

2.5.1 Applications and concerns

- **Medical diagnosis:** the app Face2Gene analyses facial photographs and, within seconds, lists candidate rare genetic syndromes. The slide's example: a clinician's hunch of Mowat–Wilson syndrome was confirmed when it appeared on the app's list.
- **Image labelling:** image acquisition → deep network → symbols (tree, grass, sky).
- **Surveillance:** the same algorithms are used for facial recognition by governments worldwide, which raises ethical questions.



2.5.2 Machine intelligence and “millions of eyeballs”

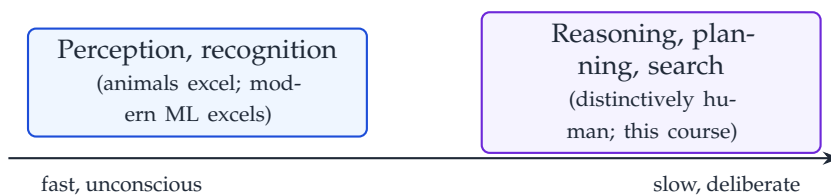
The slide places machine learning, computer vision, pattern recognition, image processing, neural networks, deep networks and speech recognition all in the *signal processing / neuro-fuzzy* layer, with symbolic reasoning above it largely untouched. The slide’s phrase “millions of eyeballs” is best read as: these systems are, in effect, vast numbers of tireless *eyes* (and ears) — cameras, microphones and sensors whose streams are turned into labels at scale. The eyeballs also run the other way: the data these systems learn from was largely labelled by very many humans. Either way, the capability sits at the perception layer, not at the reasoning layer.

2.6 Animal-like abilities vs human-level intelligence

Adnan Darwiche (CACM, 2018) argues that current ML delivers **animal-like abilities**, not human-level intelligence:

- Eagles and snakes have better vision systems than humans (a remark attributed to Judea Pearl).
- Cats have superior navigation abilities.
- Dogs recognise and react to human speech.
- African grey parrots can mimic human speech.

Yet none of these animals has the cognitive abilities typically attributed to humans. The slide’s closing line makes the point sharper: everything ML does now, humans do “in the blink of an eye”, i.e. it is fast, unconscious perception, not deliberate thought.



2.7 Performance vs competence

This argument comes from Rodney Brooks’s essay *The Seven Deadly Sins of AI Predictions* (MIT Technology Review).

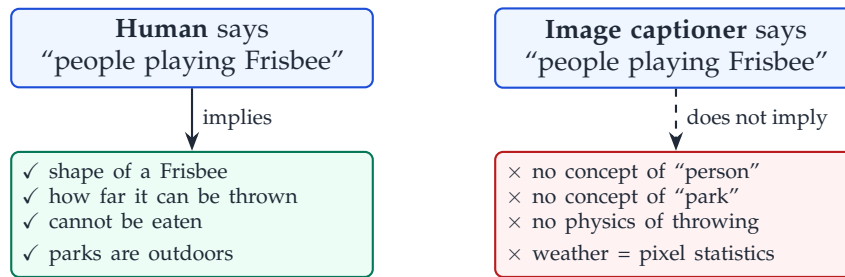
Definition: Performance vs competence

Performance is doing a specific task well (labelling a photo). **Competence** is the broad understanding we assume must lie behind that performance.

If a *person* says a photo shows people playing Frisbee in a park, we assume they can also answer: What shape is a Frisbee? Roughly how far can one be thrown? Can a person eat one? A computer that produces the same caption has no such background knowledge: it does not know what a person is, that parks are usually outdoors, that people have ages, or that weather is more than how it makes a photo look.

Intuition: The trap in our heads

Humans generalise from a person’s performance to their competence because, for humans, the two go together. Applying the same generalisation to a machine is a mistake. A system can have high performance and near-zero competence.



Note: A 2026 perspective

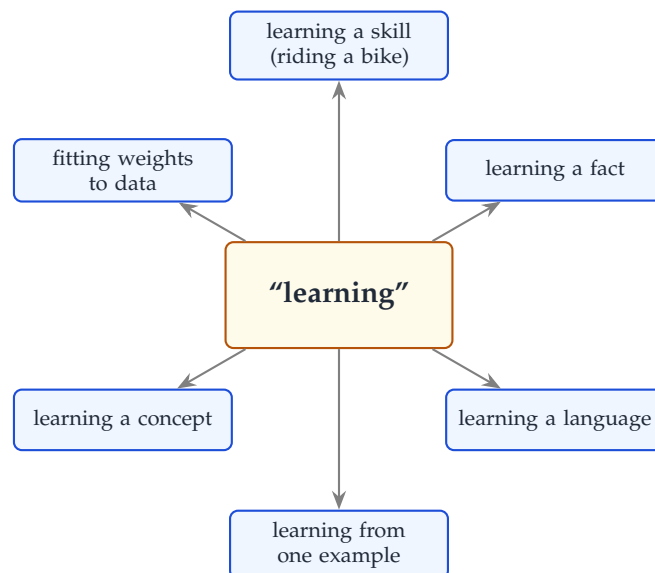
Large multimodal language models now answer many Frisbee-style questions correctly, which makes Brooks's specific example look dated. The underlying distinction still matters and is still debated: whether such systems have competence or a very broad statistical performance. For this course, what matters is the conceptual distinction and that the slides use it to motivate explicit knowledge and reasoning.

2.8 Suitcase words

Definition: Suitcase word (Minsky)

A word that packs many different meanings inside it, like a suitcase packed with different items. "Learning" is a prime example.

When people hear that "machine learning" is making great strides in some domain, they picture how a *person* would learn it. But machine learning is brittle and needs a great deal of human preparation: special-purpose code, special-purpose training data, and a custom learning structure for each new problem domain (Brooks).



Other suitcase words you will meet: *intelligence*, *understanding*, *thinking*, *consciousness*. Always ask which meaning is intended.

2.9 AlphaGo (2016): reinforcement learning in games

In 2016 DeepMind's AlphaGo beat a world champion (Lee Sedol) at Go. The slide calls it the triumph of **reinforcement learning** in games like chess and Go, and promises more on such games later in the course.

Note: Why AlphaGo belongs in a search course

AlphaGo was not pure learning. It combined deep networks (a *policy* network suggesting promising moves and a *value* network estimating who is winning) with **Monte Carlo Tree Search**. The networks act as *learned heuristics* guiding a search. This is the pattern the course teaches: search, made tractable by good heuristics. Game-tree search (Minimax, Alpha-Beta) comes later in the syllabus.

Formula / Derivation: Why Go needed more than brute-force search

If a game has branching factor b and typical length d , a full game tree has about b^d leaves.

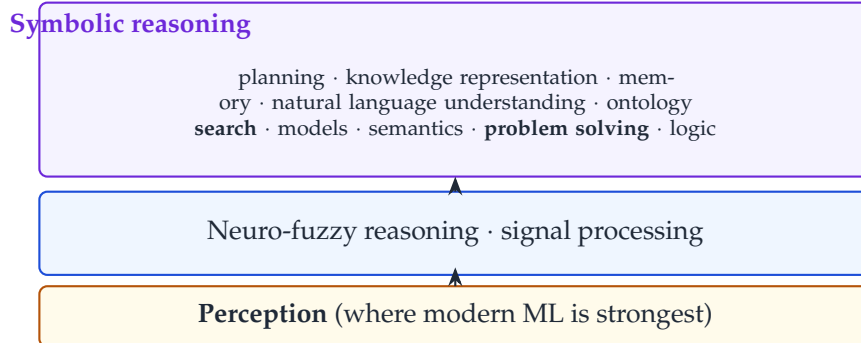
$$\text{Chess: } b \approx 35, d \approx 80 \Rightarrow 35^{80} = 10^{80 \log_{10} 35} \approx 10^{80 \times 1.544} \approx 10^{123}$$

$$\text{Go: } b \approx 250, d \approx 150 \Rightarrow 250^{150} = 10^{150 \log_{10} 250} \approx 10^{150 \times 2.398} \approx 10^{360}$$

For comparison, the observable universe has roughly 10^{80} atoms. Neither tree can be searched exhaustively; Go is far worse, which is why it fell to AI almost 20 years after chess (Deep Blue, 1997).

2.10 Human cognitive architecture

The slide stacks the layers: perception (signal processing, neuro-fuzzy reasoning) at the bottom, and symbolic reasoning at the top, containing planning, knowledge representation, memory, natural language understanding, ontology, search, models, semantics, problem solving and logic.

**Module recap**

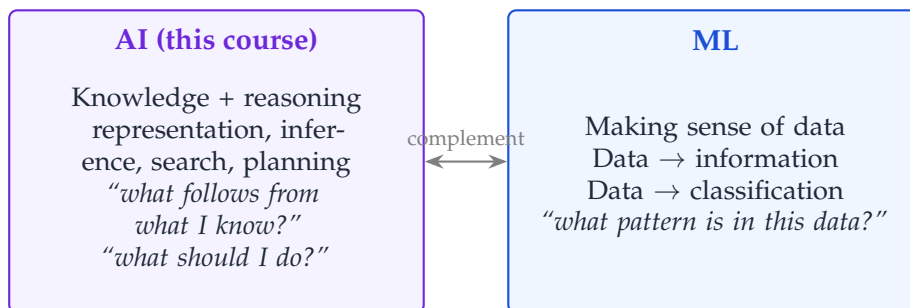
- Data + compute + better training produced the deep learning boom and the misleading idea that ML is all of AI.
- A perceptron computes $\text{step}(\mathbf{w}^\top \mathbf{x} + b)$, a single hyperplane; it cannot compute XOR (non-linearly-separable).
- Hidden layers + backprop ($\delta^{(1)} = (W^{(2)\top} \delta^{(2)}) \odot \sigma'$) learn non-linear classifiers.
- DNNs map signals to symbols. "What next??" — reasoning over those symbols is a separate problem.
- ML gives animal-like abilities; performance $\neq$ competence; "learning" is a suitcase word.
- AlphaGo = learned heuristics + tree search; b^d growth makes brute force hopeless.

Module 3 AI and ML: Knowledge, Representation and Reasoning

3.1 Two different jobs

The slide draws a sharp line:

- **AI:** symbolic knowledge representation and problem solving.
- **ML:** making sense of data. Two main flavours:
 - Data → *information*: big data, recommender systems, predictive analytics.
 - Data → *classification*: deep learning on images and language.



Intuition

ML answers “what is this?” by generalising from examples. AI (in the sense of this course) answers “given what I know, what else is true, and what should I do?” by manipulating explicit knowledge. A doctor needs both: pattern recognition to read the X-ray, reasoning to decide on treatment.

3.2 Knowledge and reasoning: the core of intelligence

Definition: The two central questions

1. What does the agent know?
2. What else does the agent know as a consequence of what it knows?

The course is concerned with **explicit declarative knowledge**, not implicit *procedural* or *tacit* knowledge.

Kind of knowledge	What it is	Example
Explicit declarative	Facts and rules written down as statements that can be inspected and reasoned about	“All humans are mortal.” “Region A is adjacent to region B.”
Procedural	Knowing <i>how</i> , embedded in a procedure	The steps of riding a bicycle; a compiled program
Tacit	Knowledge we have but cannot easily articulate	Recognising a friend’s face; a native speaker’s sense that a sentence “sounds wrong”

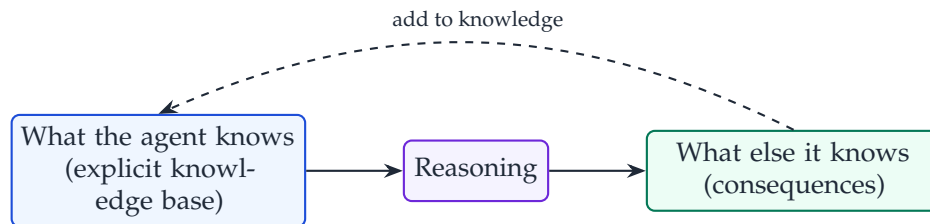
Formula / Derivation: Entailment: “what else the agent knows”

Let KB be the set of sentences the agent knows. A sentence α is a *consequence* of KB (written $KB \models \alpha$) if α is true in every situation where all of KB is true.

Example:

$$KB = \{ \text{Human}(\text{Socrates}), \forall x (\text{Human}(x) \Rightarrow \text{Mortal}(x)) \} \models \text{Mortal}(\text{Socrates})$$

The agent was never told “Socrates is mortal”, yet it *knows* it. Reasoning makes implicit knowledge explicit. (Logic and inference are the subject of the companion course on Knowledge Representation and Reasoning.)



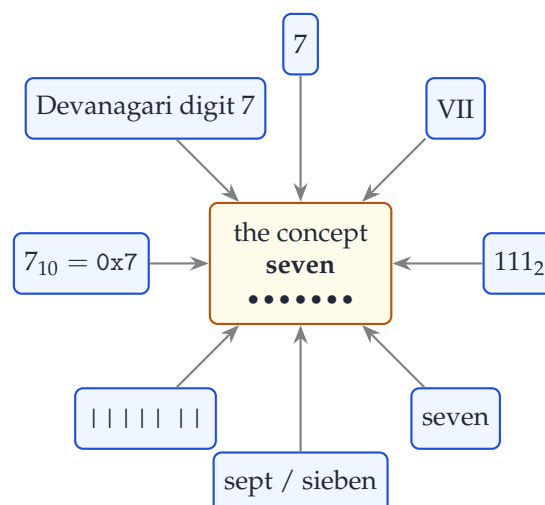
3.3 Representation and semiotics

Definition: Semiotics

The study of signs and symbols. Its core idea: **a symbol is something that stands for something else.**

Examples from the slide:

- The *number* seven can be written in many ways.
- Road signs: curves ahead, pedestrian crossing, school zone, U-turn, eating place.
- The meaning of a symbol is a **socially agreed-upon construct**. Shakespeare’s line about a rose smelling as sweet by any other name makes the point: the name is arbitrary; the thing is not.
- All languages, spoken and written, are semiotic systems.



Many symbols, one referent. The symbol is not the thing.

Formula / Derivation: Positional representation of numbers

In base β , a digit string $d_{k-1}d_{k-2} \cdots d_1d_0$ denotes

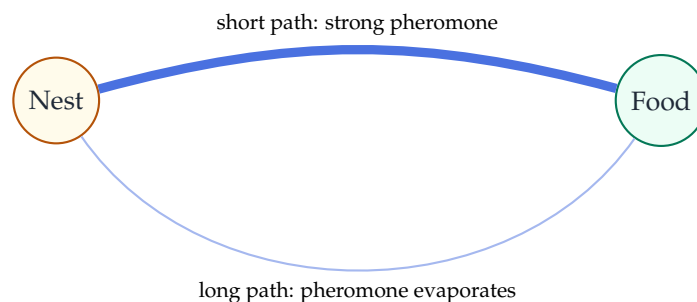
$$N = \sum_{i=0}^{k-1} d_i \beta^i, \quad 0 \leq d_i < \beta.$$

So $7 = 1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 111_2$ and $7 = 7 \cdot 10^0$. Same number, different symbol systems. The choice of representation changes *how hard reasoning is*: multiplying in Roman numerals is painful; in positional notation it is a simple algorithm. **Good representation makes reasoning easy** — a theme that returns every time we design a state space.

3.3.1 Biosemiotics**Definition: Biosemiotics**

How complex behaviour emerges when simple systems interact with each other through signs.

The slide's example is **ant pheromone trails**. No single ant knows the shortest route to food; each ant simply deposits pheromone and follows stronger trails. Shorter paths get reinforced faster (ants return sooner), so the colony as a whole “finds” short paths. This is the inspiration for **Ant Colony Optimization**, which appears later in the syllabus under population-based methods.

**3.4 Reasoning****Definition: Reasoning**

The manipulation of symbols in a meaningful manner.

Mathematics is full of algorithms we already use, and each is a reasoning procedure over symbols:

- addition and multiplication of multi-digit numbers,
- long division,
- solving systems of linear equations,
- Fourier transforms, convolution.

Formula / Derivation: Multi-digit addition as pure symbol manipulation

To add $a = \sum_i a_i 10^i$ and $b = \sum_i b_i 10^i$, process columns right to left with carry $c_0 = 0$:

$$s_i = (a_i + b_i + c_i) \bmod 10, \quad c_{i+1} = \left\lfloor \frac{a_i + b_i + c_i}{10} \right\rfloor.$$

Derivation: $a + b = \sum_i (a_i + b_i) 10^i$. Each column sum may exceed 9; write $a_i + b_i + c_i = s_i + 10c_{i+1}$, and the excess $10c_{i+1} \cdot 10^i = c_{i+1} 10^{i+1}$ moves into the next column. Nothing in the

rule refers to what the numbers *mean* (apples, rupees). The procedure operates on shapes of digits alone, yet it produces meaningful answers.

Intuition: The big idea hiding here

A clerk following the carry rule does not need to understand numbers at all; a machine can follow it too. Yet the output is *correct about the world*. This is the seed of the whole AI project: if meaningful reasoning can be done by rule-following on symbols, then machines can reason. Module 5 traces this idea from Hobbes to Newell and Simon, and Module 4 looks at the philosophers who disagree.

3.5 AI = Automation?

The slide sets three clusters side by side and asks whether AI is just automation.



Intuition: Where is the line?

- An **ATM** or **vending machine** is automation with a fixed script. No representation of goals, no alternatives considered.
- **Train reservation** or **travel planning** involves search over alternatives under constraints (seats, times, budgets). That is AI technique, even though users now just see “a website”.
- A **self-driving car** needs everything: ML to perceive, reasoning and planning to decide, control to act.

A running joke in the field: “once it works, nobody calls it AI any more.” Route planning in maps apps is A* search (later in this course), but most people think of it as mere automation.

Module recap

- AI = symbolic knowledge representation + problem solving; ML = making sense of data.
- Core of intelligence: what the agent knows, and what follows from it ($KB \models \alpha$). Focus on explicit declarative knowledge.
- A symbol stands for something else; meaning is a social convention. Choice of representation affects how easy reasoning is.
- Biosemiotics: complex behaviour from simple sign-based interaction (ant pheromones $\rightarrow$ ACO).
- Reasoning = meaningful manipulation of symbols; ordinary maths algorithms are examples.
- Automation follows fixed scripts; AI considers alternatives towards goals.

Module 4 History and Philosophy: Can Machines Think?

Two books are recommended for an insightful and entertaining account of the history and philosophy of AI: John Haugeland's *AI: The Very Idea* and Pamela McCorduck's *Machines Who Think*. Much of this module and the next two follows them.

4.1 Four definitions of AI

Who	Definition (paraphrased)	What it emphasises
Herbert Simon	A program is intelligent if it shows behaviour that we would call intelligent if a human showed it.	Behaviour: judged from outside, like the Turing test.
Barr & Feigenbaum	Physics asks what kind of place the universe is; biology asks what it means to be alive; AI asks what kind of information-processing system can ask such questions.	Self-reflection: AI as the science of the mind that does science.
Elaine Rich	AI studies techniques for solving exponentially hard problems in polynomial time by exploiting knowledge about the domain.	Complexity + knowledge: the most directly relevant to this course.
Charniak & McDermott	AI is the study of mental faculties through computational models.	Cognitive science: computers as a tool to understand minds.

Intuition: Why Elaine Rich's definition is the one to remember

Most interesting problems (scheduling, planning, puzzles, games) have search spaces that grow *exponentially* with problem size. Brute force is hopeless. Humans still solve them, because we use *knowledge* to avoid looking at most of the possibilities. Heuristic search, the heart of this course, is exactly "using knowledge to tame exponential search".

Formula / Derivation: Exponential vs polynomial: why knowledge matters

Suppose a problem has branching factor b and solution depth d . Blind search may examine

$$N_{\text{blind}} = 1 + b + b^2 + \dots + b^d = \frac{b^{d+1} - 1}{b - 1} = \Theta(b^d)$$

nodes (geometric series; derivation: let $S = \sum_{i=0}^d b^i$, then $bS - S = b^{d+1} - 1$).

A perfect heuristic that always picks the right move examines only bd nodes (it looks at b children at each of d levels) — *linear* in d . Real heuristics land somewhere in between; their quality is measured by an *effective branching factor* b^* with $N \approx (b^*)^d$, and a good heuristic drives b^* close to 1.

Example: $b = 10$, $d = 10$: blind $\approx 1.1 \times 10^{10}$ nodes; perfect heuristic = 100 nodes.

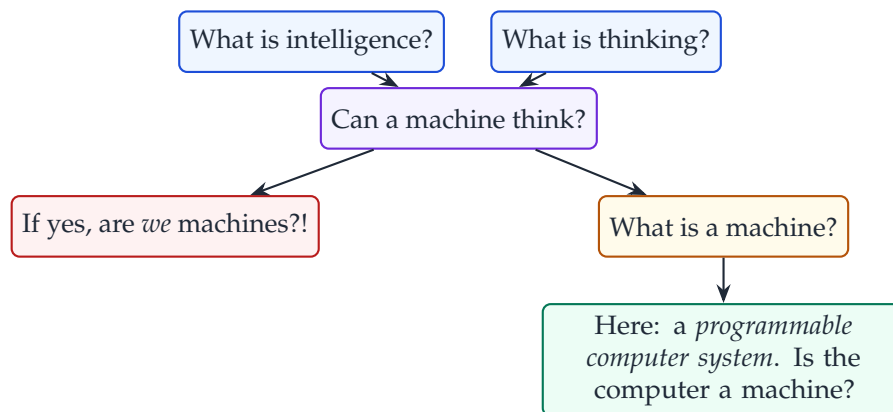
4.2 Machines with minds of their own

Haugeland's position (paraphrased from *AI: The Very Idea*): AI does not aim to *mimic* intelligence or produce a clever fake. It wants the genuine article — machines with minds in the full, literal sense. The theoretical basis is the daring claim that **we are, at root, computers ourselves**: that thinking and computing are fundamentally the same thing.

Note: Strong vs weak AI

This is what philosophers call **strong AI** (a suitably programmed computer really *has* a mind) as opposed to **weak AI** (computers are useful tools for *simulating* or studying minds). Haugeland describes strong AI; Charniak & McDermott's definition is closer to weak AI.

4.3 The fundamental questions



Intuition: Why “is the computer a machine?” is a real question

A steam engine does one thing. A programmable computer can become *any* machine you can describe precisely (a calculator, a chess player, a word processor). That universality (formalised by Turing) is why the question “can a machine think?” became serious only after computers existed: if thinking is describable precisely, a computer can do it.

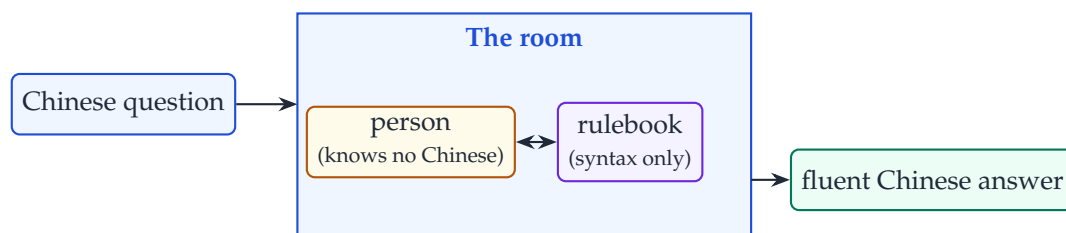
4.4 The raging debates over thinking machines

Critic	Argument	Typical response
Dreyfus	Intelligence depends on unconscious instincts that can never be captured in formal rules.	Partly vindicated: hand-written rules did prove brittle, and learned (non-rule) systems succeeded at perception. But this does not show machines <i>cannot</i> have such abilities, only that rules alone may not suffice.
Searle	Chinese Room: a person in a room follows syntactic rules to answer Chinese questions without understanding Chinese. So symbol manipulation is not understanding.	“Systems reply”: the <i>room as a whole</i> (person + rulebook) understands. Also, how many rules would the room need? The slide raises this: the rulebook would be astronomically large, which weakens our intuition about the thought experiment.
Penrose	Something quantum-mechanical happens in the brain that current physics cannot explain, so brains are not computers.	Speculative; no accepted evidence that brain function depends on such effects.
Others	Emotion, intuition, consciousness, ethics.	Open questions; many are suitcase words (Module 2).

Exam trap

The slide writes “Herbert Dreyfus”; the philosopher is **Hubert** Dreyfus (author of *What Computers Can't Do*, 1972). Herbert is Simon's first name, which is probably the source of the slip.

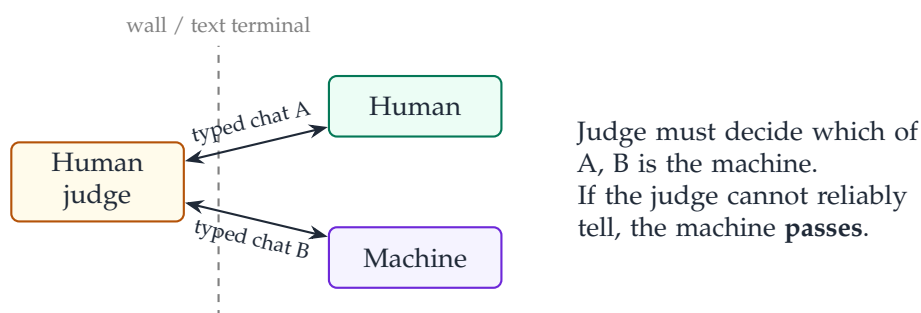
4.4.1 The Chinese Room in a diagram



Searle: the person understands nothing, so neither does a computer running the same rules.
 Systems reply: understanding (if any) belongs to the whole system, not the component.

4.5 Alan Turing's imitation game

Alan Turing (1912–1954) thought the question “can machines think?” was too meaningless to deserve discussion as stated. In his 1950 paper *Computing Machinery and Intelligence* (Mind, 59, 433–460) he replaced it with a test he called the **Imitation Game**, now known as the **Turing Test**.

**Definition: Turing test**

A human judge holds text conversations with two hidden parties, one human and one machine. If, after conversation, the judge cannot reliably distinguish the machine from the human, the machine is said to exhibit intelligent behaviour.

Intuition: What the test does and does not measure

It is an *operational* definition, like Simon's: judge by behaviour, not by internal mechanism. Its strength is that it avoids arguing about the meaning of "think". Its weakness, shown by the next few slides, is that clever *tricks* (deflection, humour, personality) can fool judges without much intelligence underneath.

Formula / Derivation: When is "cannot reliably tell" true? A statistical view

If the judge is guessing, each verdict is correct with probability $p = 0.5$. Over n independent trials, the number of correct identifications $X \sim \text{Binomial}(n, 0.5)$:

$$P(X \geq k) = \sum_{j=k}^n \binom{n}{j} 0.5^n.$$

If judges identify the machine correctly far more than $n/2$ times, it clearly fails. Turing's own 1950 prediction: by around 2000, an average interrogator would have no more than a 70% chance of correct identification after five minutes of questioning (i.e. the machine fools judges at least 30% of the time).

4.6 The Loebner Prize and chatbots

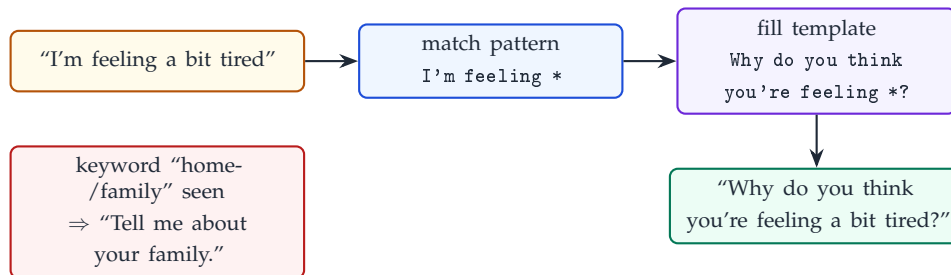
The Loebner Prize was an annual Turing-test competition. The slide shows excerpts from the 2013 leader, a bot called **Izar**. What to notice in the transcript:

- It greets the judge by name with fake enthusiasm ("It's about time you contacted me!").
- It leaks an internal error message ("Too much recursion in AIML") — AIML is the pattern-matching rule language it was written in.
- When asked its favourite fruit, it produces incoherent template output with unfilled slots ("Tell me more about your unknown").
- When asked about music, it gives a quirky, specific answer (Mongolian throat singing) and turns the question back on the judge — a *distracting technique*.

The slide adds that nowadays we have Alexa, chatbots and GPT-3 (and, since the slides were made, far more capable LLMs).

4.7 ELIZA (1966)

ELIZA was a simple natural language program written at MIT by **Joseph Weizenbaum** around 1966. It used simple rules to transform the user's input into its response. Its most famous script, **DOCTOR**, imitated a Rogerian psychotherapist (a style of therapy that reflects the patient's words back). The name comes from Eliza Doolittle, the working-class character taught to speak "properly" in George Bernard Shaw's play *Pygmalion*.



Formula / Derivation: ELIZA as a rule system

Each rule is a pair (*decomposition pattern, reassembly template*), plus pronoun swapping ($I \leftrightarrow you$, $my \leftrightarrow your$):

* I'm feeling X $\rightarrow$ Why do you think you're feeling swap(X)?

Keywords have priorities; if nothing matches, a content-free fallback is used ("Please go on."). There is *no model of the world and no memory of meaning*; the program never knows what "tired" is.

The human reaction: people, including (per the slide) a Russian scientist chatting with DOCTOR at Stanford, confided in it as if it understood. Weizenbaum found this disturbing and wrote *Computer Power and Human Reason: From Judgment to Calculation* (1976) about the limits of computers and the danger of attributing understanding to them.

Note: The ELIZA effect

The human tendency to attribute understanding to systems that produce fluent language is now called the **ELIZA effect**. It is the conversational version of the performance-vs-competence confusion from Module 2.

4.8 Conversations with machines: the problem

Given web-scale data, corpus-based natural language processing and clever distraction techniques, it has become easy to build chatbots that are *impressive but lack intelligence*. The Turing test rewards deception. **Hector J. Levesque** therefore proposed a new test.

4.9 Winograd schemas: an alternative Turing test

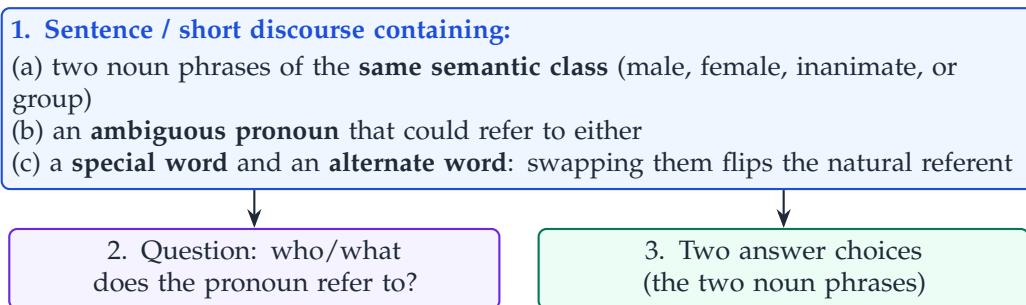
Definition: Winograd Schema Challenge

A multiple-choice test of pronoun resolution (*anaphora resolution*) that requires commonsense knowledge of the situation described. Each question has exactly two possible answers, so it is a **binary decision problem**.

4.9.1 Why it is harder to game

- Each question is pointed and requires knowledge of the subject matter.
- The two versions of a sentence differ by one word (e.g. *give* vs *receive*) whose contexts are statistically very similar in text, yet the correct answer flips.
- So having a huge text corpus “would likely not help much” — the test is designed to be **Google-proof**.
- The claim: doing better than chance requires actually figuring out what is going on.

4.9.2 Structure of a schema

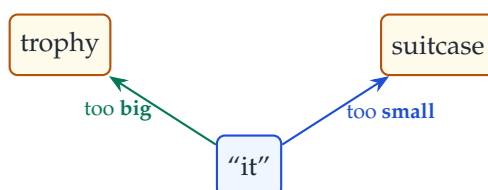


Intuition: Why “same semantic class”?

If one noun were “the trophy” and the other “Mary”, the pronoun “it” vs “she” would give the answer away grammatically. Keeping both nouns in the same class forces the solver to use *meaning*, not grammar.

4.9.3 The four examples, solved

Sentence (special / alternate word in bold)	Referent	Knowledge needed
<i>Example 1 (Winograd, 1972 — the original)</i>		
The city councilmen refused the demonstrators a permit because they feared violence.	councilmen	Authorities refuse permits to <i>prevent</i> violence they fear.
...because they advocated violence.	demonstrators	People who advocate violence are the ones denied permission.
<i>Example 2 (IJCAI 2016 challenge)</i>		
John took the water bottle out of the backpack so that it would be lighter .	backpack	Removing something makes the <i>container</i> lighter.
...so that it would be handy .	bottle	You take out the thing you want to have at hand.
<i>Example 3</i>		
The trophy would not fit in the brown suitcase because it was too small .	suitcase	A container that is too small cannot hold an object.
...because it was too big .	trophy	An object that is too big cannot go into a container.
<i>Example 4</i>		
The lawyer asked the witness a question, but he was reluctant to repeat it.	lawyer	The asker is the one who would repeat a question.
...reluctant to answer it.	witness	The person asked is the one who answers.



One word changes the answer. Word statistics of “big” and “small” near “trophy” and “suitcase” are similar; spatial commonsense is not.

Note: What happened next

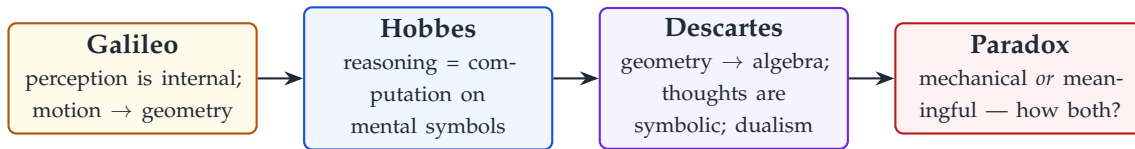
Levesque, Davis and Morgenstern formalised the challenge around 2011–12. For years systems scored near chance. Around 2019–20, large pre-trained language models reached human-level scores on the larger WinoGrande benchmark, and the challenge was widely declared “solved” — though researchers argue about whether models exploit subtle dataset artefacts rather than truly reasoning. This is itself a lesson: any fixed benchmark eventually gets optimised against.

Module recap

- Definitions of AI: behavioural (Simon), self-reflective (Barr & Feigenbaum), knowledge-vs-complexity (Rich), cognitive (Charniak & McDermott).
- Blind search costs $\Theta(b^d)$; knowledge (heuristics) is what brings it down.
- Critics: Dreyfus (unformalisable instinct), Searle (Chinese Room: syntax $\neq$ semantics), Penrose (quantum brain).
- Turing replaced “can machines think?” with the Imitation Game; it can be gamed by tricks (Loebner/Izar, ELIZA).
- Winograd schemas: binary pronoun-resolution questions, same-class nouns, one special word flips the answer; designed to be Google-proof and to require commonsense.

Module 5 Minds and Machines Made of Matter

This module traces one philosophical thread: the idea that **thinking is the manipulation of symbols**, and therefore something a mechanism might do. The thread runs Galileo → Hobbes → Descartes and ends in a paradox that AI claims to resolve.



5.1 Galileo: perception is an internal process

Galileo Galilei (born 15 February 1564, Pisa). In *The Assayer* (1623) he argued that tastes, odours and colours are, as far as the object is concerned, mere names; they exist in consciousness. Remove the living creature and these qualities vanish.

Intuition: Primary vs secondary qualities

The object has shape, size, motion (“primary” qualities). “Red” or “sweet” (“secondary” qualities) are how *we* react to the particles and their motion. So what we perceive is already an *internal representation*, not the world itself. This is the first step towards the idea that the mind works with its own symbols.

Galileo also wrote that the book of the universe is written in the language of mathematics, whose characters are triangles, circles and other geometric figures. He showed that **geometry could represent and reason about motion**.

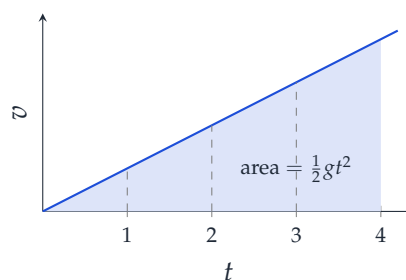
Formula / Derivation: Galileo’s insight in modern form: motion as geometry

For a body starting at rest with constant acceleration g , velocity grows linearly: $v(t) = gt$. On a velocity–time graph this is a straight line, and the *distance travelled is the area of the triangle* under it:

$$s(t) = \frac{1}{2} \cdot \underbrace{t}_{\text{base}} \cdot \underbrace{gt}_{\text{height}} = \frac{1}{2}gt^2 \quad [\text{s} \cdot \text{m s}^{-2} \cdot \text{s} = \text{m}]$$

Consequence (Galileo’s odd-number law): distances covered in successive equal time intervals are in ratio $1 : 3 : 5 : 7 : \dots$, because $s(n) - s(n-1) \propto n^2 - (n-1)^2 = 2n - 1$.

The point for AI: a *physical process* (falling) can be faithfully represented by *manipulating figures* (triangles), and conclusions drawn from the figures are true of the world.



5.2 Thomas Hobbes: the grandfather of AI

Thomas Hobbes (born 5 April 1588, Westport near Malmesbury, England) was the first to put forward the view that **thinking is the manipulation of symbols**.

- Galileo: all reality is mathematical; everything is made of particles, and sensing a smell or taste is how we react to those particles.
- Hobbes extended this: *thought itself* is made up of (expressed in) particles that the thinker manipulates.
- Just as geometry can represent motion, thinking can be done by manipulating mental symbols.

Definition: Reasoning = computation (Hobbes, 1655)

Hobbes wrote that by reasoning he meant computation, and that to compute is to add things together or to find what remains when one thing is taken from another. Hence **to reason is the same as to add or subtract**.

Intuition: Adding and subtracting ideas

Hobbes meant it broadly: “man” = “rational” + “animal”; to see that “a man is an animal”, subtract “rational” from the idea of man. Concepts are built by combining simpler parts, and reasoning takes them apart and puts them together. Modern echoes: description logics that define concepts as intersections, and even vector arithmetic on word embeddings (“king – man + woman $\approx$ queen”).

Fun fact from the slide: Bill Watterson named the tiger in *Calvin and Hobbes* after him.

5.3 René Descartes: thoughts are symbols

René Descartes (1596–1650, born La Haye en Touraine, France).

- Animals are wonderful machines. Human beings are too, *except that they possess a mind*.
- Galileo: motion $\rightarrow$ geometry. Descartes: **geometry** $\rightarrow$ **algebra** (analytic / coordinate geometry).
- Descartes: thoughts themselves are symbolic representations.
- Slogan: everything is “applied maths” — even thought.



Formula / Derivation: Geometry $\rightarrow$ algebra: an example of representation change

A circle of radius r centred at (a, b) : the set of points at distance r from the centre. Using Pythagoras on the right triangle with legs $x - a$ and $y - b$:

$$(x - a)^2 + (y - b)^2 = r^2 \quad [\text{length}^2 = \text{length}^2]$$

Now a geometric question — “where does the line $y = mx + c$ meet the circle?” — becomes pure symbol manipulation: substitute and solve the quadratic

$$(1 + m^2)x^2 + 2(m(c - b) - a)x + (a^2 + (c - b)^2 - r^2) = 0,$$

whose discriminant tells you (0, 1 or 2 intersections) without drawing anything. The *meaning* (circles, lines) is carried along by *rule-following* on symbols.

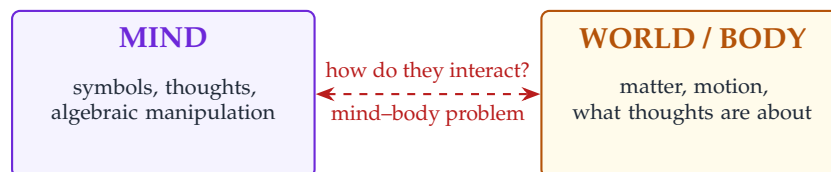
5.4 Dualism: mind and body

Descartes held that a symbol and what it symbolises are two different things:

$$\underbrace{\text{algebraic manipulation}}_{\text{thinking}} \neq \underbrace{\text{what it is about}}_{\text{subject of thought}} \quad \text{mind} \neq \text{world}.$$

This separation raises three questions (Haugeland):

1. What makes a notation *suitable* for symbolising?
2. What makes a suitable notation *actually* symbolise (have meaning)?
3. How can thought and matter interact? — the **mind-body problem**.



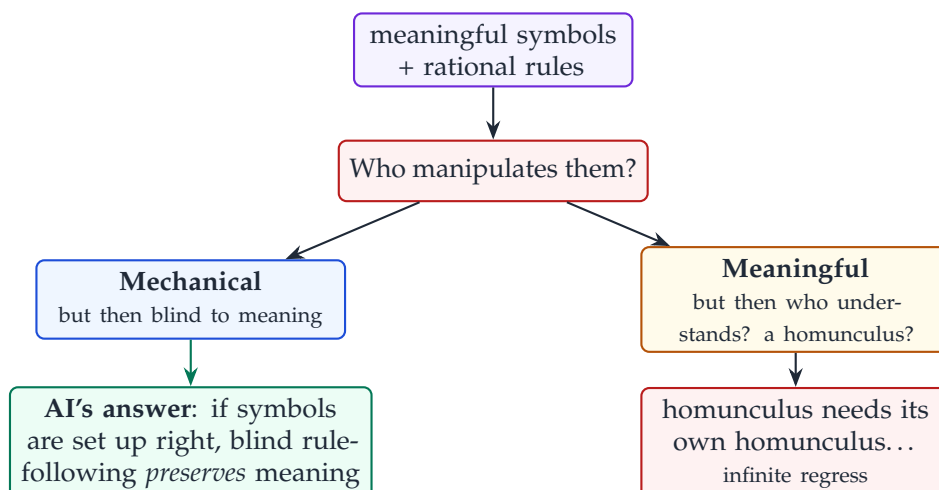
5.5 The paradox of mechanical reason

Formula / Derivation: The paradox, stated as an argument

If reasoning is the manipulation of *meaningful* symbols according to *rational* rules, **then** who is doing the manipulating?

- If the manipulator is **mechanical**, it follows rules blindly and cannot “pay attention” to meaning.
- If the manipulator **attends to meaning**, it seems to need a mind of its own — a faculty of will, a transcendental ego, or a **homunculus** (“little man” inside the head).

It seems it can be mechanical or meaningful, but how can it be both?



Intuition: How AI claims to dissolve the paradox

Look back at the carry rule (Module 3) or the circle equation. The rules operate on shapes of symbols only — purely mechanical — yet, because the symbols are set up to correspond to numbers or geometry, the results *remain meaningful*. Haugeland’s slogan for this: “**if you take care of the syntax, the semantics will take care of itself.**” A computer is a machine that

manipulates symbols mechanically in a way that respects their meaning. That is why it counts as a candidate for a mind. Whether this fully answers Searle (Module 4) is still debated.

Note: Further reading suggested on the slide

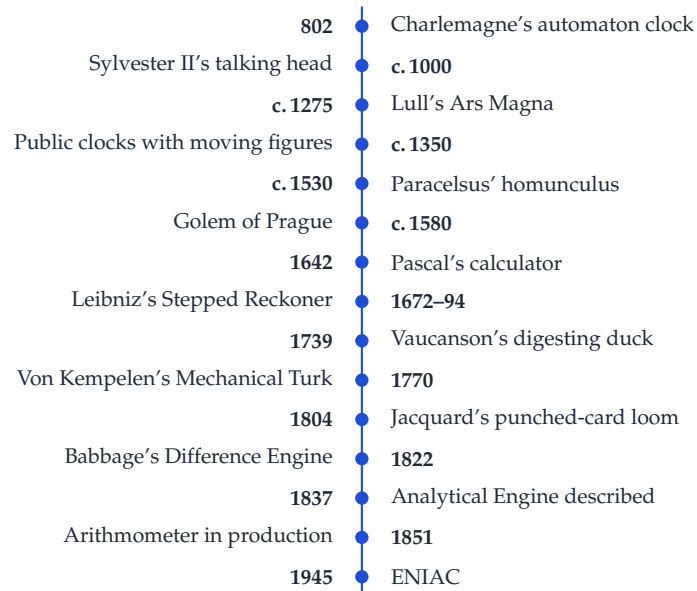
Douglas Hofstadter's *Gödel, Escher, Bach*; Hofstadter & Dennett's *The Mind's I*; Hofstadter's *I Am a Strange Loop*. They explore how meaning and selfhood might emerge from mechanical, self-referential symbol systems.

Module recap

- Galileo: perceived qualities are internal; geometry can represent and reason about motion ($s = \frac{1}{2}gt^2$ as triangle area).
- Hobbes: reasoning is computation — adding and subtracting (mental) symbols. “Grandfather of AI”.
- Descartes: geometry $\rightarrow$ algebra; thoughts are symbolic; dualism (mind $\neq$ world) $\rightarrow$ mind-body problem.
- Paradox of mechanical reason: mechanical *or* meaningful? AI's answer: well-designed formal systems make mechanical manipulation meaning-preserving.

Module 6 Artificial People and Real Mechanisms

The dream of artificial beings is ancient; the *machinery* that made it plausible arrived slowly, through clocks, automata and calculators. This module walks from myth to the first electronic computer.



Timeline of mechanisms (evenly spaced, not to scale; mythological figures are older and undated)

6.1 Artificial people in Western mythology and fiction

Figure	Story
Hephaestus: Talos	In Greek myth (the slide cites Homer's <i>Iliad</i>), the smith-god Hephaestus built Talos, a bronze man who patrolled the beaches of Crete.
Hephaestus: Pandora	Commissioned by Zeus to punish mankind for accepting Prometheus's gift of fire; overcome by curiosity, she opens the infamous casket ("Pandora's box"), releasing evils into the world.
Pygmalion & Galatea	Pygmalion, disappointed by real women, carved Galatea in ivory; Aphrodite brought her to life. (Shaw's <i>Pygmalion</i> , source of ELIZA's name, borrows this title.)
Daedalus	Credited with lifelike statues that wheezed, blinked and scuttled about.
Pope Sylvester II (946–1003)	Said to have made a talking head with a limited vocabulary that predicted the future by answering yes or no.
Paracelsus (1493–1541)	Physician said to have created a <i>homunculus</i> (little man); he spoke of duplicating God's greatest miracle, the creation of man.
Judah Loew ben Bezalel (1520–1609)	In Jewish folklore, sculpted a Golem , an animated being made from clay, to defend the Jews of Prague; only he could instruct it.

Intuition: Why myths belong in an AI course

They show that the questions are old: Can we make a being that acts on its own? Who controls it? (The Golem obeys only its maker; Pandora's curiosity causes disaster.) These are early forms of today's concerns about autonomy and control of AI systems.

6.2 Real mechanisms: clocks, automata and thinking machines

- **802 AD:** Caliph Haroun al-Rashid reportedly gave Emperor Charlemagne an elaborate clock that sent a dozen horsemen out of a dozen windows at noon and brought them back.
- **Mid-14th century:** large clocks with moving figures became public monuments (Strasbourg, Nuremberg, Lübeck and Berne followed Italian cities), and *talking brass heads* became associated with learned men.
- The **Archbishop of Salzburg** had a working model of an entire miniature town, driven by water power from a nearby stream.

6.2.1 The zairja and Lull's Ars Magna

Arab astrologers are credited with a "thinking machine" called the **zairja**, designed to generate ideas mechanically using a technique of "breaking down" (*al-jabr*, from which "algebra" comes). By combining number values associated with letters and categories, it produced new lines of thought (McCorduck).

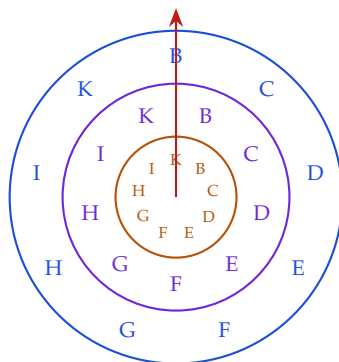
The Catalan missionary **Ramon Lull** (1232–1315) built a Christian version, the **Ars Magna**, which aimed to bring reason to bear on every subject and arrive at truth *without the trouble of thinking or fact-finding*.

Formula / Derivation: Lull's wheels: mechanical combinatorics

Lull's device used concentric paper discs, each inscribed with a set of concepts (e.g. 9 divine attributes labelled B–K). Rotating the discs lines up combinations.

- With k discs of n symbols each, the number of alignments is n^k (e.g. $9^3 = 729$).
- Unordered pairs from 9 concepts: $\binom{9}{2} = \frac{9 \cdot 8}{2} = 36$.

This is the first recorded attempt to **systematically enumerate a space of possibilities** to find truths — a primitive form of generate-and-test search.



Rotate the discs; read off combinations along a radius. Each alignment is one "question" to consider. Generate-and-test, 700 years before computers.

Note: The zairja and search

The zairja and Ars Magna are "thinking machines" only in a loose sense: they *generate* candidates mechanically, but a human must *test* which combinations are meaningful. Search algorithms need both a generator (successor function) and a tester (goal test). You will meet this split

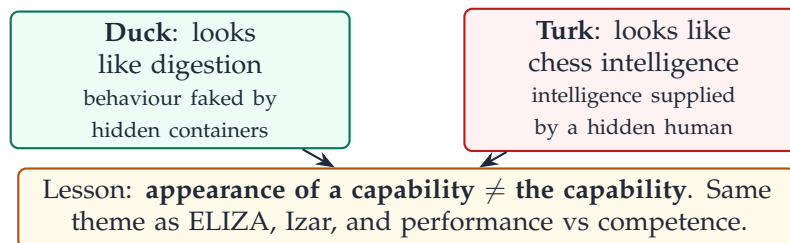
formally in Week 2.

6.3 Vaucanson's Duck (1739)

Jacques de Vaucanson (1709–1782), French inventor, built several automata. His most famous, the **Digesting Duck**, appeared to drink, eat, quack, splash, and digest grain. In fact no digestion happened: food went into one container, and pre-stored droppings were released from a second. Vaucanson hoped a truly digesting automaton could one day be built.

6.4 Von Kempelen's Chess-Playing Turk (1770)

Wolfgang von Kempelen (1734–1804) unveiled the **Mechanical Turk** in 1770 to impress Empress Maria Theresa of Austria. It appeared to play strong chess and could perform the *knight's tour*. It was an illusion: a human chess master hid inside. It beat many challengers, reportedly including Napoleon and Benjamin Franklin. Edgar Allan Poe's essay *Maelzel's Chess Player* (1836) argued it was a fraud.



Note

Amazon named its crowdsourcing platform *Mechanical Turk* after this device: “artificial artificial intelligence”, i.e. humans hidden behind an API. Many ML datasets were labelled this way — one reading of the “millions of eyeballs” remark in Module 2.

Formula / Derivation: The knight's tour as a search problem (preview)

Find a sequence of knight moves visiting all 64 squares exactly once. A state = (current square, set of visited squares). A knight has at most 8 moves, so a naive upper bound on sequences is $8^{63} \approx 10^{57}$. Brute force is hopeless; the classic **Warnsdorff heuristic** (always move to the square with the fewest onward moves) finds tours almost instantly. A 1770s party trick is a good example of heuristic search.

6.5 Mechanical arithmetic

6.5.1 Pascal's calculator (1642)

Blaise Pascal (1623–1662, born Clermont-Ferrand) invented a mechanical calculator using lantern gears in 1642, building about 50 prototypes before presenting it publicly in 1645. Called the Arithmetic Machine, Pascal's Calculator, and later the **Pascaline**, it could **add and subtract directly**, and multiply and divide by repetition. By 1654 about twenty had been sold, but cost and complexity limited sales and production stopped.

Formula / Derivation: How a Pascaline subtracts using only addition: nines' complement

The Pascaline's wheels only turned one way (adding). Subtraction used complements. For n -digit numbers:

$$a - b = a + \underbrace{(10^n - 1 - b)}_{\text{nines' complement}} + 1 - 10^n.$$

Example ($n = 3$): $523 - 187$. Nines' complement of 187 is $999 - 187 = 812$. $523 + 812 = 1335$; add 1: 1336; drop the leading 1 (subtract 10^3): **336**. Check: $523 - 187 = 336$. ✓

Every digit of the nines' complement is just $9 - d_i$, which a machine can display by reading the wheel from the other side. **A new representation turns a hard operation into an easy one.**

6.5.2 Leibniz's Stepped Reckoner

Gottfried Wilhelm von Leibniz (1646–1716, born Leipzig) began his own calculator after Pascal's death. He invented the **Leibniz wheel** (stepped drum) in 1673; it remained in use for three centuries until electronic calculators in the mid-1970s. The **Stepped Reckoner** (designed c. 1672, completed 1694) did multiplication by repeated addition and division by repeated subtraction, on 8-digit numbers.

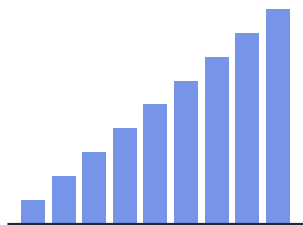
Formula / Derivation: Multiplication by repeated addition with shifting

Naively, $a \times b$ needs b additions. With a movable carriage (shift = multiply by 10), write $b = \sum_i b_i 10^i$:

$$a \times b = \sum_i b_i (a \cdot 10^i),$$

so you add a (shifted i places) exactly b_i times for each digit. Number of additions = $\sum_i b_i \leq 9 \times (\text{number of digits of } b)$ instead of b .

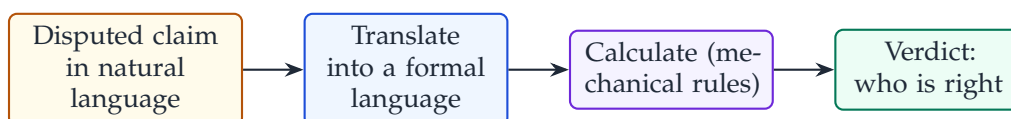
Example: 347×26 : add 347 six times (2082), shift, add 347 twice ($2 \times 3470 = 6940$). Total $2082 + 6940 = \mathbf{9022}$ using 8 additions instead of 26.



Leibniz stepped drum (unrolled): nine ridges of increasing length. A sliding gear placed at position k meets k ridges per rotation, so one turn adds digit k . Setting a digit becomes a *position*; adding it becomes a *rotation*.

6.6 Leibniz's *calculus ratiocinator*

Leibniz believed much of human reasoning could be reduced to calculation, and that such calculation could settle disputes. He argued that the only way to fix our reasonings is to make them as tangible as mathematicians' so errors show at a glance; then, in a dispute, people could simply say "*Calculemus*" — let us calculate — and see who is right.

**Intuition**

This is the dream behind formal logic, automated theorem proving, and every rule-based expert system. The Logic Theorist (Module 7) is its first real implementation. Its limits were shown by

Gödel and Turing in the 1930s: some true statements cannot be derived by any fixed formal system, and some questions cannot be decided by any algorithm.

6.7 Commercial success: the Arithmometer

The **Arithmometer**, patented in France by Thomas de Colmar in 1820 and manufactured 1851–1915, was the first mechanical calculator strong and reliable enough for daily office use. It drove the shift from *human computers* (people employed to calculate) to calculating machines in the second half of the 19th century.

6.8 Charles Babbage: the first computer

Charles Babbage (1791–1871, born London): mathematician, philosopher, inventor and mechanical engineer, remembered for originating the idea of a **programmable computer**. As a child he was fascinated by automata shown by a man called Merlin (one walked and bowed with graceful motions; another danced).

6.8.1 The Difference Engine (1822)

Babbage started in 1822 with the **Difference Engine**, designed to compute values of **polynomial functions** automatically. The first design had about 25,000 parts, weighed about 15 tons (13,600 kg) and stood 8 ft (2.4 m) high. The London Science Museum later built one from his designs; his son assembled part of one from parts in his lab.

Formula / Derivation: The method of finite differences — why polynomials need only addition

Define the forward difference $\Delta p(x) = p(x+1) - p(x)$, and $\Delta^k = \Delta(\Delta^{k-1})$.

Claim: if p has degree n , then Δp has degree $n-1$, so $\Delta^n p$ is constant and $\Delta^{n+1} p = 0$.

Proof: for $p(x) = a_n x^n + \dots$,

$$\Delta p(x) = a_n [(x+1)^n - x^n] + (\text{lower-degree terms}) = a_n [nx^{n-1} + \dots] + \dots$$

by the binomial theorem; the x^n terms cancel, leaving degree $n-1$ with leading coefficient na_n . Repeating n times gives the constant $\Delta^n p = n! a_n$. ■

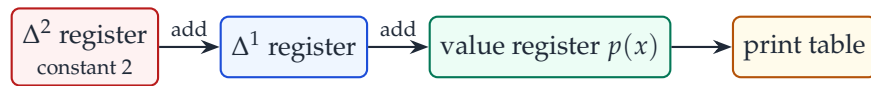
Consequence: after setting up the first value and first n differences, every further value is produced by **additions only** (no multiplication), cascading from the constant difference upwards:

$$\Delta^k p(x+1) = \Delta^k p(x) + \Delta^{k+1} p(x), \quad k = n-1, \dots, 0.$$

Worked table for $p(x) = x^2 + x + 41$ (a polynomial Babbage actually used in demonstrations; it produces primes for $x = 0, \dots, 39$). Degree 2, so $\Delta^2 p = 2! \cdot 1 = 2$:

x	0	1	2	3	4	5	6	7
$p(x)$	41	43	47	53	61	71	83	97
Δp	2	4	6	8	10	12	14	
$\Delta^2 p$	2	2	2	2	2	2		

To get $p(8)$: new $\Delta p = 14 + 2 = 16$; $p(8) = 97 + 16 = \mathbf{113}$. Check: $64 + 8 + 41 = 113$. ✓



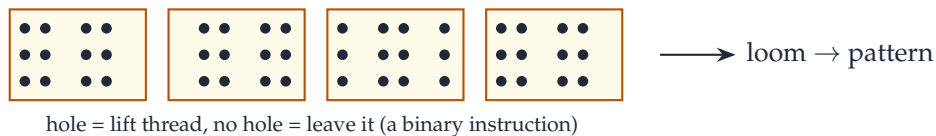
one crank turn = one row of the table

Intuition: Why this mattered

In the 1800s, navigation, engineering and finance depended on printed tables (logarithms, trig functions) computed by human computers and full of errors. Many useful functions can be approximated piecewise by polynomials, and polynomials need only repeated addition, which gears do well. Babbage reduced a “thinking” task to a mechanical cascade of additions.

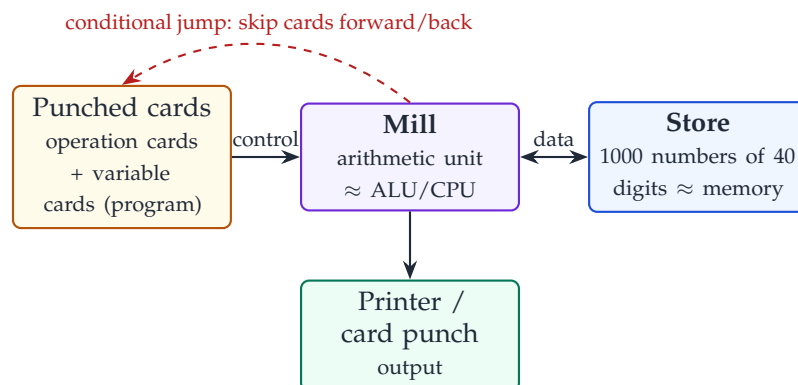
6.8.2 Jacquard looms: programs on punched cards

The **Jacquard loom** (1804) used chains of punched cards to control which threads were lifted, so the same loom could weave different patterns by changing the cards. **The card chain is a program; the loom is the machine that runs it.** Babbage borrowed the idea.



6.8.3 The Analytical Engine (1837)

The **Analytical Engine**, first described in 1837, was a proposed *general-purpose* mechanical computer. It had an arithmetic logic unit, control flow with **conditional branching and loops**, and integrated **memory** — the first design that, in modern terms, is **Turing-complete**. Babbage built only a trial part (on display at the Science Museum, London).



Note: Mapping to a modern computer

Store → RAM; Mill → ALU/CPU; cards → program; the ability to move backwards or forwards through the card chain depending on a result → **branches and loops**. Branching is what makes a machine general-purpose: without it, a machine can only run a fixed sequence.

6.9 Ada Lovelace: the first programmer

Augusta Ada King, Countess of Lovelace (1815–1852, born Augusta Ada Byron in London), wrote extensive notes on the Analytical Engine. They contain what is recognised as the first algorithm intended to be carried out by a machine (computing Bernoulli numbers), so she is often called the world’s first computer programmer. The programming language **Ada** is named after her.

6.9.1 Beyond number crunching

Lovelace saw that the Engine's potential went far beyond arithmetic. Paraphrasing her notes: it could act on things other than numbers, provided their mutual relations could be expressed in the abstract science of operations. For example, if the relations of pitched sounds in harmony and composition could be so expressed, the Engine might compose elaborate pieces of music of any complexity.

Intuition: Lovelace's leap = the symbol idea again

Numbers are just one kind of symbol. If *anything* — notes, words, chess positions — can be encoded as symbols with rules, a general machine can manipulate it. This is the step from “calculator” to “symbol processor”, and it is exactly the Physical Symbol System Hypothesis (Module 7) a century early.

Note: Lady Lovelace's objection

Lovelace also wrote that the Engine has no pretensions to *originate* anything; it can only do what we know how to order it to do. Turing addressed this in his 1950 paper as “Lady Lovelace's Objection”, answering that machines can surprise their makers.

6.10 ENIAC: the first electronic computer

ENIAC (1945) was huge by modern standards:

Component	Count / size
Vacuum tubes	17,468
Crystal diodes	7,200
Relays	1,500
Resistors	70,000
Capacitors	10,000
Hand-soldered joints	about 5 million
Weight	27 tons
Dimensions	about 2.4 m × 0.9 m × 30 m
Floor area	167 m ²
Power	150 kW

Formula / Derivation: Putting ENIAC's numbers in perspective

Power per vacuum tube (upper bound, if all power went to tubes):

$$\frac{150\,000\text{ W}}{17\,468} \approx 8.6\text{ W per tube.}$$

Energy per day: 150 kW × 24 h = 3600 kWh.

ENIAC did about 5,000 additions per second. A modern GPU does on the order of 10^{15} operations per second at a few hundred watts, i.e. roughly $10^{15}/5 \times 10^3 = 2 \times 10^{11}$ times more operations per second, on about 1/500 of the power (150 kW vs ~300 W).

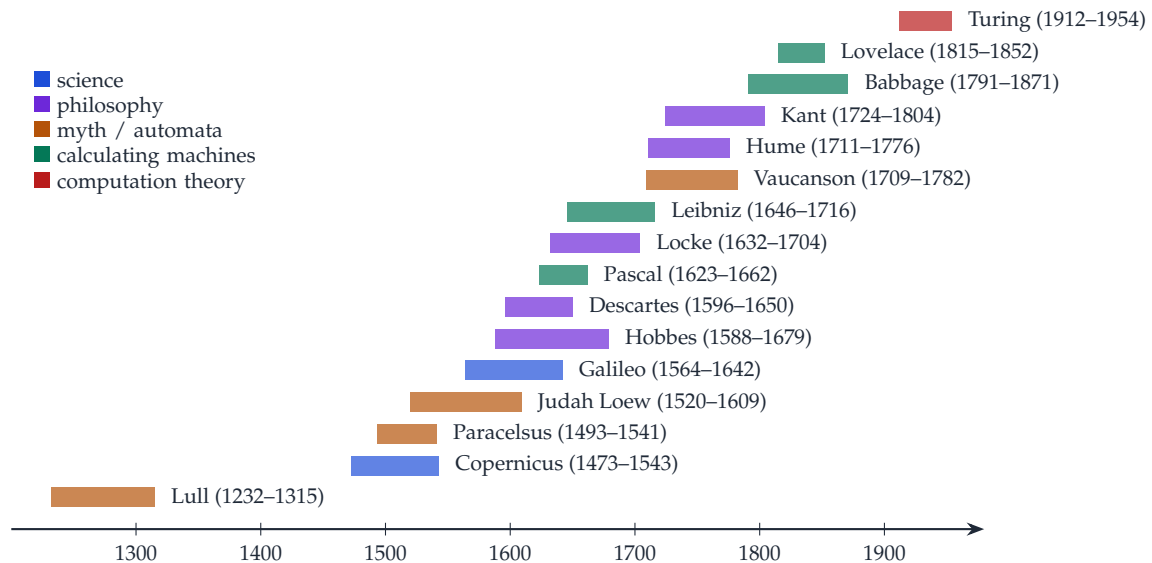
Module recap

- Myths (Talos, Pandora, Galatea, Golem) show the dream of artificial beings is ancient.
- Clocks and automata (Charlemagne's clock, Salzburg town, Vaucanson's duck) showed mechanisms could imitate life — sometimes by trickery (duck, Turk).
- Zairja and Lull's Ars Magna: mechanical combination of concepts — primitive generate-and-test.
- Pascal (add/subtract, complements), Leibniz (stepped drum, repeated addition; *calculus*), de Colmar (Arithmometer, office use).
- Babbage: Difference Engine (polynomials via finite differences, only additions); Analytical Engine (store, mill, cards, branching $\Rightarrow$ Turing-complete), inspired by Jacquard's punched cards.
- Lovelace: first published machine algorithm; machines can manipulate any symbols, not just numbers.
- ENIAC: first electronic computer, 17,468 tubes, 150 kW.

Module 7 Modern Times: The Birth of AI

7.1 The cast of characters so far

The slide places the thinkers and inventors on a timeline from 1300 to 1900 (plus Turing). It also adds philosophers not otherwise discussed: **John Locke** (1632–1704), **David Hume** (1711–1776) and **Immanuel Kant** (1724–1804), who argued about where knowledge comes from (experience vs reason) — questions AI inherits.



7.2 How AI got its name: Dartmouth, 1956

The name **artificial intelligence** is credited to **John McCarthy**, who with Marvin Minsky and Claude Shannon (and Nathaniel Rochester) organised the **Dartmouth Conference** in 1956. It was proposed as a two-month, ten-person study, based on the conjecture that every aspect of learning, or any other feature of intelligence, can in principle be described so precisely that a machine can be made to simulate it. (McCorduck, *Machines Who Think*, ch. 5, has the full story.)

Intuition: The Dartmouth conjecture is a bet on symbols

“Described so precisely that a machine can simulate it” is the Hobbes–Leibniz–Lovelace idea turned into a research programme: if intelligence can be *written down* as rules over symbols, computers (which now existed) can run those rules.

7.2.1 The organisers

Organiser	Contribution
John McCarthy (1927–2011)	Assistant professor at Dartmouth then. Designed Lisp , the language of AI research for decades. Worked on logic and commonsense reasoning.
Marvin Minsky (1927–2016)	Then a Harvard Junior Fellow; one of the most influential figures in AI. Co-founded the MIT AI Lab with McCarthy. Known for frames (structured knowledge representation). Wrote <i>The Society of Mind</i> and <i>The Emotion Machine</i> .
Nathaniel Rochester (1919–2001)	Young IBM engineer; designed the IBM 701 and wrote the first assembler . Supervised Arthur Samuel 's checkers program. Reportedly IBM stopped AI work after marketing found people were frightened of “electronic brains”.
Claude Shannon (1916–2001)	Bell Labs mathematician, already famous for information theory . Had hired McCarthy and Minsky as summer students in 1952.

Exam trap

The slide calls Minsky's book “Society of the Mind”; the actual title is *The Society of Mind* (1986). Similarly the slide spells Newell's first name “Alan”; it is **Allen** Newell (not to be confused with Alan Turing).

7.2.2 The show stealers: Newell and Simon

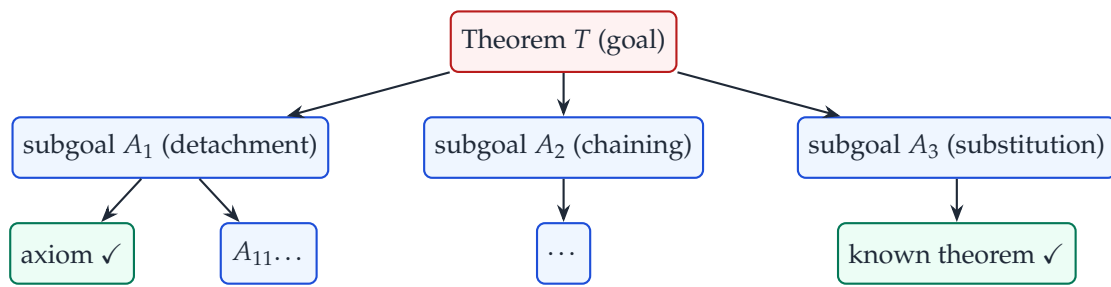
Herbert Simon and Allen Newell were little-known researchers at Carnegie Tech and RAND, invited almost as an afterthought (McCorduck). Yet, with **J. C. Shaw** of RAND, they had already built a working program: the **Logic Theorist (LT)**. It was the first program deliberately engineered to mimic human problem-solving skills. It proved many theorems from Russell and Whitehead's *Principia Mathematica*, finding shorter and more elegant proofs for some.

Formula / Derivation: How the Logic Theorist searched

LT worked **backwards** from the theorem to be proved, towards axioms and already-proved theorems, using inference rules:

- **Detachment** (modus ponens backwards): to prove B , find a known $A \Rightarrow B$ and set subgoal A .
- **Substitution**: specialise a known theorem by replacing variables.
- **Chaining**: to prove $A \Rightarrow C$, find known $A \Rightarrow B$ and set subgoal $B \Rightarrow C$ (or vice versa).

This creates a **search tree of subgoals**. Without guidance it explodes; LT used *heuristics* (e.g. prefer subgoals that look similar to known theorems) — the first heuristic search program. This course's backward-chaining and goal-tree methods (AO*, goal stack planning) descend from it.



7.3 Simon and Newell

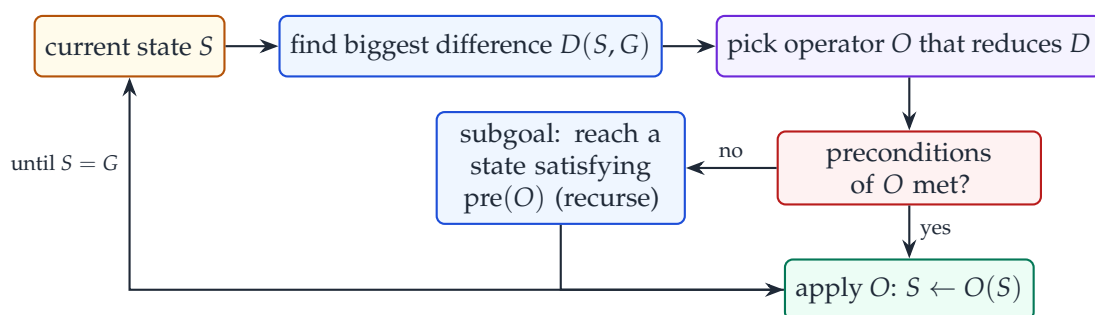
- **Herbert Simon** (1916–2001): an American polymath at Carnegie Mellon, working across cognitive psychology, computer science, economics, management, public administration, philosophy of science, sociology and political science. (He won the Turing Award with Newell in 1975 and the Nobel Prize in Economics in 1978 for work on decision-making and *bounded rationality*.)
- **Allen Newell** (1927–1992): Simon’s long-term collaborator at CMU. Designed **IPL** (Information Processing Language), in which LT was written.

7.4 The information-processing view: GPS and means–ends analysis

Simon and Newell became leading figures in AI and built a strong group at CMU. Their **General Problem Solver (GPS)** pioneered heuristics in search and used a human-like strategy called **means–ends analysis**. Their work defined the **information-processing approach** to AI. A later example from CMU is the **SOAR** cognitive architecture developed by John Laird (with Newell and Rosenbloom).

Definition: Means–ends analysis

Repeatedly: (1) compare the current state with the goal and find the most important **difference**; (2) choose an **operator** known to reduce that difference; (3) if the operator’s preconditions are not met, set up achieving them as a **subgoal** and recurse; (4) apply the operator and repeat until no difference remains.



Formula / Derivation: Worked example: getting to a conference in another city

Operators with preconditions:

Operator	Reduces difference	Precondition
Fly	distance > 500 km	at airport, have ticket
Take cab	distance 5–50 km	have cab booked
Walk	distance < 1 km	—
Book (ticket / cab)	“do not have X”	have phone

Run (home in Bengaluru $\rightarrow$ venue in Delhi):

1. Biggest difference: about 1,700 km $\Rightarrow$ choose *Fly*. Preconditions fail (not at airport, no ticket) $\Rightarrow$ subgoals.
2. Subgoal “have ticket” $\Rightarrow$ *Book ticket* (precondition: have phone $\checkmark$).
3. Subgoal “at airport” (distance $\approx$ 35 km) $\Rightarrow$ *Take cab*; subgoal “cab booked” $\Rightarrow$ *Book cab*.
4. Apply *Fly*. New biggest difference: airport $\rightarrow$ venue ($\approx$ 20 km) $\Rightarrow$ Take cab (book first).
5. Last difference < 1 km $\Rightarrow$ Walk. Difference is zero: done.

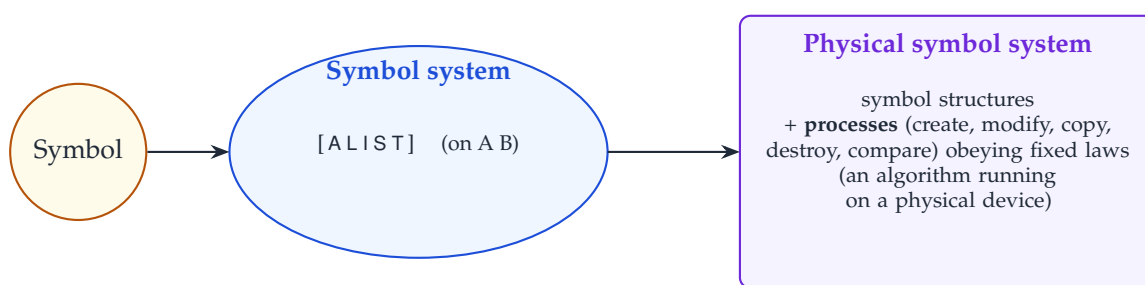
Note the **order**: MEA attacks the *largest* difference first and fills in details later — exactly how people plan. It is also the ancestor of STRIPS-style planning and goal stack planning later in this course.

Note: Weakness of means–ends analysis

It is greedy about differences. Some problems require temporarily *increasing* a difference (e.g. in the Tower of Hanoi you must move small discs out of the way, away from the goal). MEA can get stuck; systematic search does not.

7.5 Physical symbol systems

Term	Meaning	Examples
Symbol	A perceptible something that stands for something else.	Letters, numerals, road signs, musical notation
Symbol system	A collection of symbols forming a pattern (a structure).	Words, arrays, lists, even a tune
Physical symbol system	A symbol system that obeys laws of some kind — a formal system with processes that create and transform symbol structures.	Long division, an abacus, an algorithm (running on a computer or a brain)



Definition: The Physical Symbol System Hypothesis (Newell & Simon, 1976)

A physical symbol system has the **necessary and sufficient** means for general intelligent action.

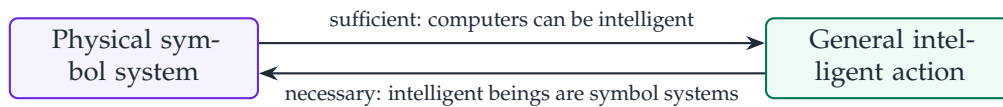
Formula / Derivation: Unpacking “necessary and sufficient”

Let $P = \text{“X is a physical symbol system”}$ and $I = \text{“X is capable of general intelligent action”}$. The hypothesis claims $P \Leftrightarrow I$ (in the sense of having the means):

- **Sufficient** ($P \Rightarrow I$): any sufficiently large, suitably organised PSS *can* be made intelligent $\Rightarrow$ computers can in principle be intelligent.

- **Necessary** ($I \Rightarrow P$): anything intelligent (including humans) *must be* a PSS $\Rightarrow$ human minds are symbol processors.

The two halves are independent claims. Connectionists and critics like Dreyfus attack *necessity* (intelligence might not be symbolic at all); Searle attacks *sufficiency* (symbol manipulation is not enough for understanding).



This view — intelligence as the ability to manipulate symbols — is called **symbolic AI** or **classical AI**. Haugeland named it **GOFAI**: “Good Old-Fashioned Artificial Intelligence”.

Note: Where this course stands

Search algorithms are the purest form of GOFAI: states are symbol structures, operators are processes that transform them, and a goal test is a symbolic comparison. Modern systems (AlphaGo, LLM agents with planners) often combine learned (sub-symbolic) components with symbolic search — sometimes called *neuro-symbolic AI*.

Module recap

- Dartmouth 1956 (McCarthy, Minsky, Shannon, Rochester) named the field; the conjecture: every feature of intelligence can be described precisely enough to simulate.
- McCarthy: Lisp, logic. Minsky: MIT AI Lab, frames. Rochester: IBM 701, first assembler, Samuel’s checkers. Shannon: information theory.
- Newell, Simon & Shaw’s Logic Theorist: first program to mimic human problem solving; backward search over subgoals with heuristics.
- GPS used means–ends analysis: reduce the biggest difference, recursively achieving operator preconditions.
- PSSH: a physical symbol system is necessary and sufficient for general intelligence $\Rightarrow$ symbolic AI / GOFAI.

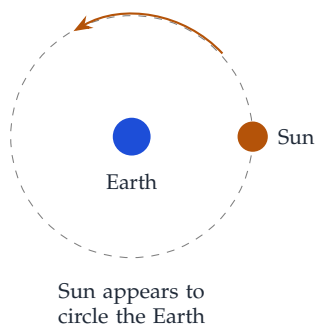
Module 8 Reality, Complexity and the Domains of Reasoning

This module asks a question every AI system must answer implicitly: **what should the elements of my world model be?** The answer — choose the right level of abstraction — is what makes search possible at all.

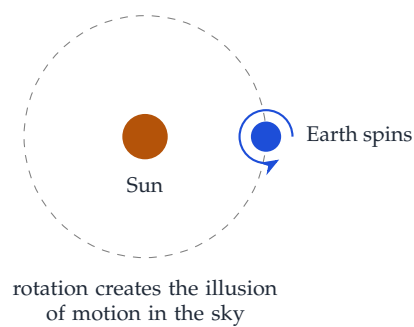
8.1 What we see is not what really is: Copernicus

Nicolaus Copernicus (born 19 February 1473, Toruń, Royal Prussia, Kingdom of Poland). In *On the Revolutions of the Celestial Spheres* he argued that the Sun, Moon and stars only *appear* to move across the sky; the appearance is created by the rotating Earth.

What we see (geocentric)



What is (heliocentric)



Intuition

Perception gives a *model*, not reality (the same point Galileo made about colour and taste). Science is the process of building better models. An AI agent is in the same position: it only ever reasons about its internal representation.

8.2 Reality

What is really out there? One thing is clear: everything in the physical world is made of a small number of **fundamental particles** (even though physics has not settled exactly what they are). The laws of physics that explain the behaviour of these particles are, in principle, sufficient to explain the behaviour of any *ensemble* of them — unless there is no matter at all (the position called **idealism**).

Note: Reductionism

The claim that the behaviour of wholes follows from the laws governing their parts is *reductionism*. The slide accepts it *in principle*. The next slide shows why it is useless *in practice*.

8.3 Complexity

A human adult contains roughly 10^{27} atoms, continuously interacting with countless surrounding atoms: the air we breathe, the food we eat, the vibrating air molecules we hear as sound, and the trillions of photons (which have momentum but no mass) striking us. Could we write down and solve the equations for all of these? And if we did, what would we get — a prediction of where each of 10^{27} atoms goes?

Formula / Derivation: Estimating the number of atoms in a human body

Model a 70 kg adult as roughly all water (about 60% is water; the rest is mostly C, H, O, N compounds with similar atoms-per-gram, so the estimate holds to an order of magnitude).

$$\begin{aligned}\text{moles of H}_2\text{O} &= \frac{70\,000\text{ g}}{18\text{ g/mol}} \approx 3.9 \times 10^3\text{ mol} && [\text{g} / (\text{g mol}^{-1}) = \text{mol}] \\ \text{molecules} &= 3.9 \times 10^3 \times 6.022 \times 10^{23} \approx 2.3 \times 10^{27} && [\text{mol} \times \text{mol}^{-1}] \\ \text{atoms} &= 3 \times 2.3 \times 10^{27} \approx 7 \times 10^{27} && [3\text{ atoms per molecule}]\end{aligned}$$

So “about 10^{27} ” is right to within an order of magnitude ($7 \times 10^{27} \approx 10^{27.8}$).

Formula / Derivation: Why simulating at the particle level is hopeless

Suppose we could spend just **one** floating-point operation per atom per femtosecond (10^{-15} s, roughly the timescale of atomic vibrations). To simulate **one second** of a human:

$$10^{27}\text{ atoms} \times 10^{15}\text{ steps} = 10^{42}\text{ operations.}$$

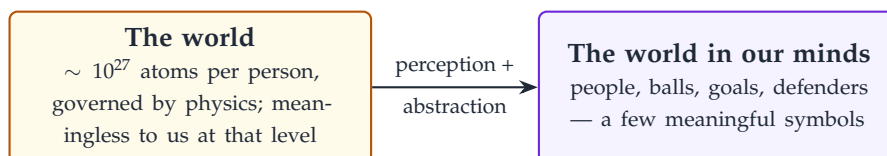
A top supercomputer does about 10^{18} operations per second (exascale), so this takes

$$\frac{10^{42}}{10^{18}} = 10^{24}\text{ s} \approx \frac{10^{24}}{3.15 \times 10^7}\text{ years} \approx 3 \times 10^{16}\text{ years,}$$

about two million times the age of the universe (1.4×10^{10} years). And one flop per atom-step is absurdly optimistic.

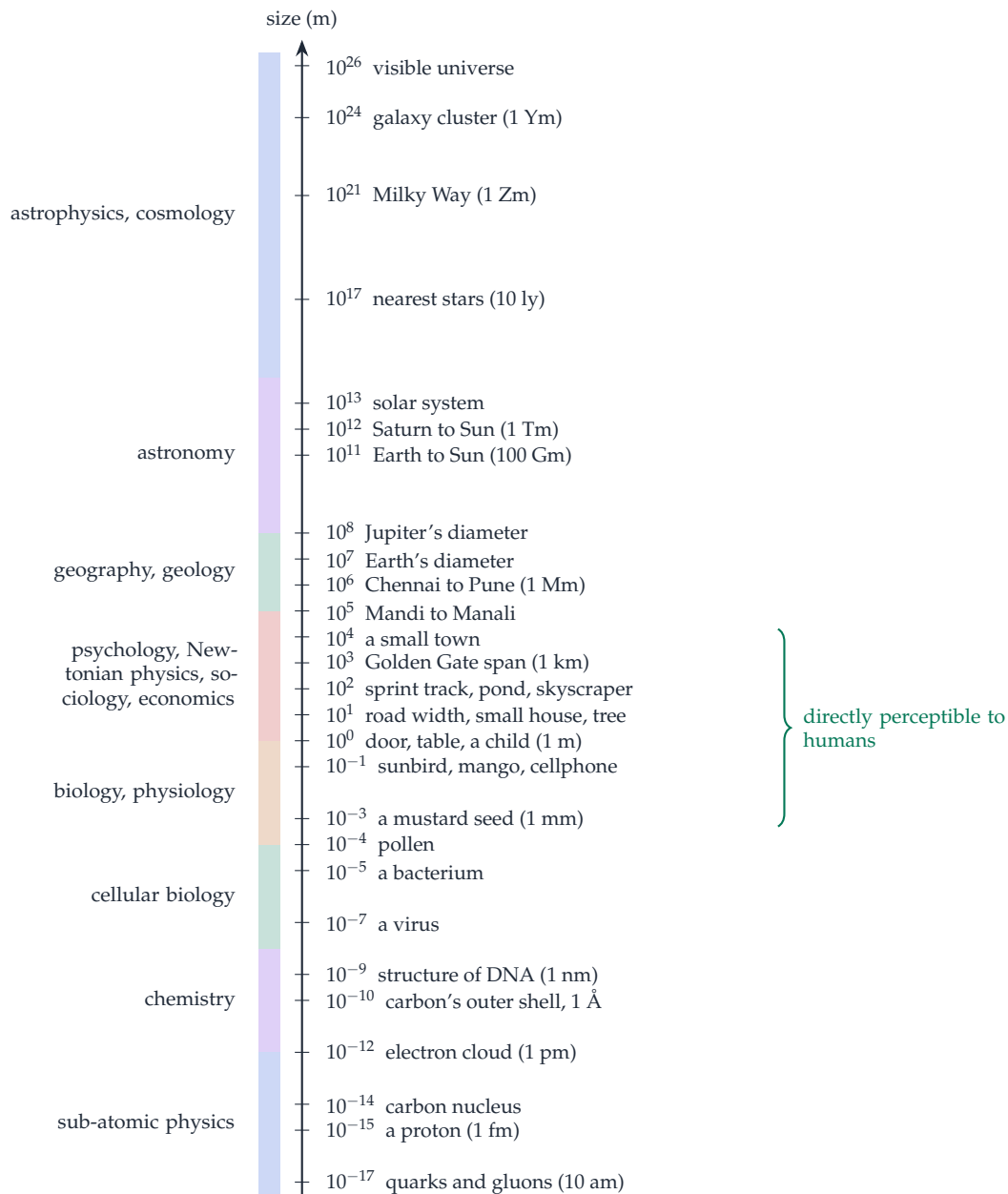
8.4 The world in our minds

The world, including us, runs on the fundamental laws of physics; nothing else is needed. But we, as thinking creatures, build our own worlds in our minds, and *only our own creation is meaningful to us*. The slide mentions films that play with this idea: *The Matrix*, *Inception*.



8.5 Powers of ten

The slides reproduce a scale ladder (from *Quarks to Quasars*, inspired by Charles and Ray Eames’s 1977 film *Powers of Ten*):



Note: Unit prefixes used on the slides

atto (a) 10^{-18}	femto (f) 10^{-15}	pico (p) 10^{-12}	nano (n) 10^{-9}
micro (μ) 10^{-6}	milli (m) 10^{-3}	kilo (k) 10^3	mega (M) 10^6
giga (G) 10^9	tera (T) 10^{12}	peta (P) 10^{15}	zetta (Z) 10^{21} , yotta (Y) 10^{24}

(Exa 10^{18} sits between peta and zetta. The slide's entries like "100 gigameters = 10^{11} m" combine a prefix with a multiplier.)

Formula / Derivation: Reading the ladder

From the proton (10^{-15} m) to the visible universe (10^{26} m) is

$$\frac{10^{26}}{10^{-15}} = 10^{41},$$

i.e. 41 orders of magnitude. The geometric middle of the ladder is $\sqrt{10^{-15} \cdot 10^{26}} = 10^{5.5}$ m $\approx$ 300 km; humans ($\approx$ 1 m = 10^0) sit about 5.5 orders of magnitude below it — 15 orders above the

proton, 26 below the visible universe. We are *near* the middle on a log scale, but not at it. Check a slide entry: Earth–Sun distance $149\,600\,000\text{ km} = 1.496 \times 10^8\text{ km} = 1.496 \times 10^{11}\text{ m}$, so “100 gigameters (10^{11} m)” is the right order of magnitude.

8.5.1 Our perceptible universe

The “Our Perceptible Universe” slide highlights the middle band — roughly from a mustard seed (10^{-3} m) to a few kilometres — that humans perceive directly. Everything outside it we know only through instruments and theory. **Scientific progress lets us extend our concepts to different scales.**

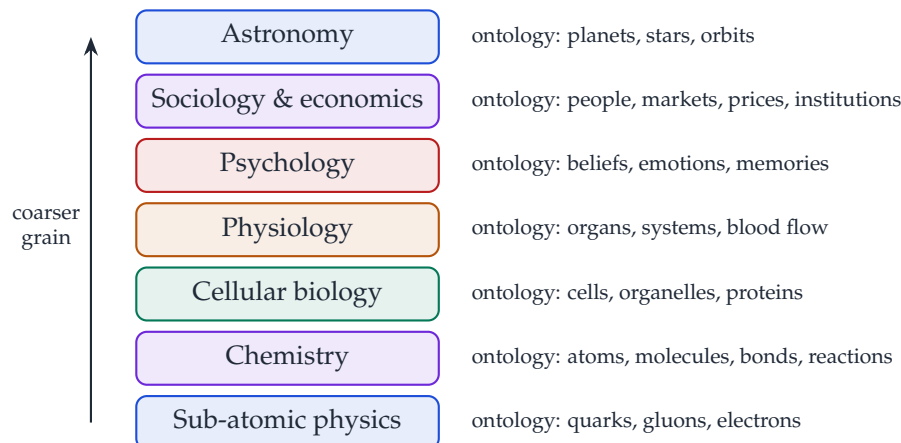
8.6 The domains for reasoning

Definition: Key principle

The real world is made of fundamental particles, but those particles **cannot be the elements of the domain** we use for representation and reasoning: there are far too many of them, even to describe a grain of rice. Depending on the focus of study, we represent atoms, molecules, cells, organs, creatures, or societies. **Every domain of study defines its own ontology.**

Definition: Ontology (in AI)

The set of **kinds of things** (and relations between them) that a representation assumes exist. The ontology fixes the vocabulary of the domain: what can be talked about and reasoned over.



Intuition: Why this matters for search

When you set up a search problem you are choosing an ontology. For route finding: cities and roads (not asphalt molecules). For chess: 64 squares and pieces (not the wood). For the 8-puzzle: tile positions (not the plastic). The **right abstraction keeps the state space small** enough to search. Choose too fine a grain and even trivial problems become intractable; choose too coarse and you cannot express the solution.

Formula / Derivation: Abstraction shrinks the state space — a toy calculation

Describe a football pitch position:

- **Fine grain:** 22 players, each at one of 1 cm grid cells on a $100\text{ m} \times 64\text{ m}$ pitch: $10^4 \times 6.4 \times 10^3 = 6.4 \times 10^7$ cells. Configurations $\approx (6.4 \times 10^7)^{22} \approx 10^{172}$.
- **Coarse grain:** divide the pitch into $10 \times 6 = 60$ zones. Configurations $\approx 60^{22} \approx 10^{39}$.
- **Task-relevant grain:** “which teammates are unmarked and closer to goal than me?” — a

handful of boolean facts, maybe $2^{10} \approx 10^3$ situations.
The decision a striker needs (next module) lives at the last level.

Module recap

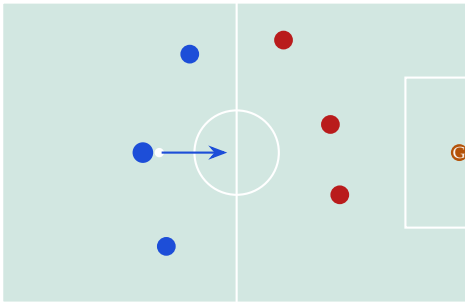
- Copernicus: appearances (the sky moving) differ from reality (Earth rotating); perception yields models.
- Reality is made of fundamental particles and, in principle, physics explains everything — but a human has $\sim 10^{27}$ atoms, and particle-level simulation is utterly infeasible.
- Thinking creatures build a meaningful world in their minds; only that is meaningful to us.
- Powers of ten: 41 orders of magnitude from protons to the universe; each scale has its own science.
- Every domain of study defines its own ontology. Choosing the right level of abstraction is what makes representation and search feasible.

Module 9 Problem Solving: Where Search Fits

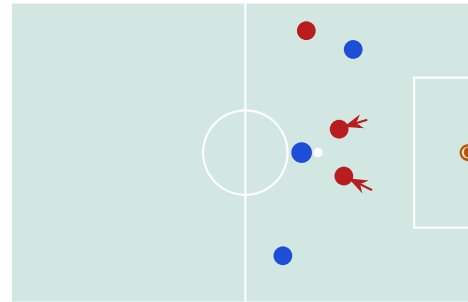
9.1 A striker on a football field

The slides tell a story in frames. You are a striker for the blue team, moving forward with the ball. Defenders close in. What should you do?

Frame 1: moving forward

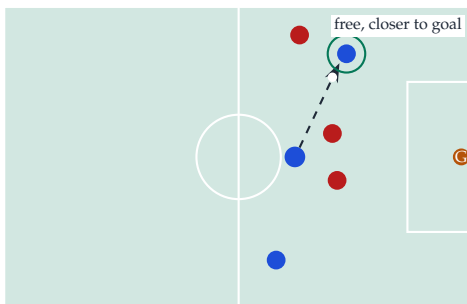


Frame 2: defenders close in

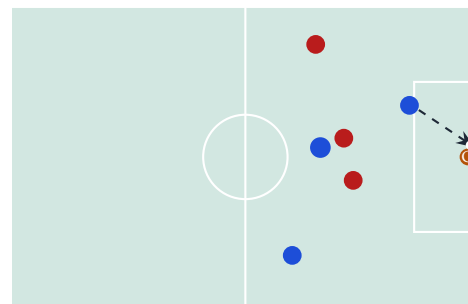


What should you do?

Frame 3: pass to the free teammate



Frame 4: better chance to score



Intuition: What the story is teaching

An intelligent agent does not keep running into defenders. It **imagines** alternatives (dribble, shoot, pass left, pass right), **evaluates** each against the goal (scoring), and picks the best: pass to a teammate who is free and has a better chance to score. That imagine-evaluate-choose loop is problem solving by search.

9.2 Problem solving defined

Definition: Problem solving

An **autonomous agent** in some **world** has a **goal** to achieve and a **set of actions** to choose from in striving for the goal.

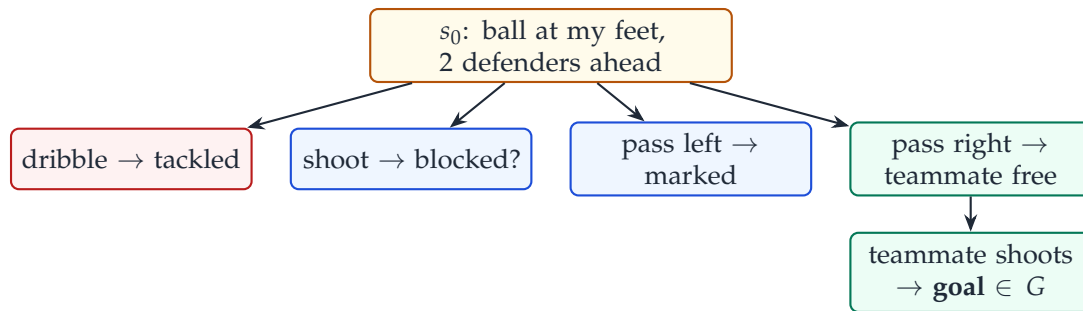
Formula / Derivation: The ingredients of a search problem (preview of Week 2)

A problem can be specified as a tuple $\langle S, s_0, A, T, G \rangle$:

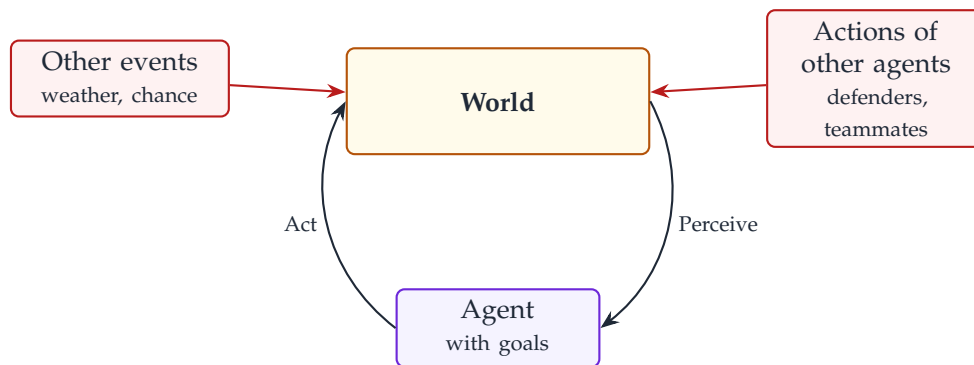
- S : set of **states** (the agent's representation of the world),
- $s_0 \in S$: **initial state**,
- $A(s)$: **actions** applicable in state s ,
- $T(s, a) = s'$: **transition** (successor) function — the result of doing a in s ,

- $G \subseteq S$ (or a goal test): the **goal** states.

A **solution** is a sequence of actions $a_1, \dots, a_k$ with $T(\dots T(T(s_0, a_1), a_2) \dots, a_k) \in G$. Searching means exploring sequences in the model until one reaches G .



9.3 The real world is complex



The real world changes by itself (other events) and because of other agents (the defenders have their own goals). Our knowledge is incomplete; actions can fail (a pass can be intercepted).

9.4 Learn to walk before you run: simplifying assumptions

The course starts with simple problems where:

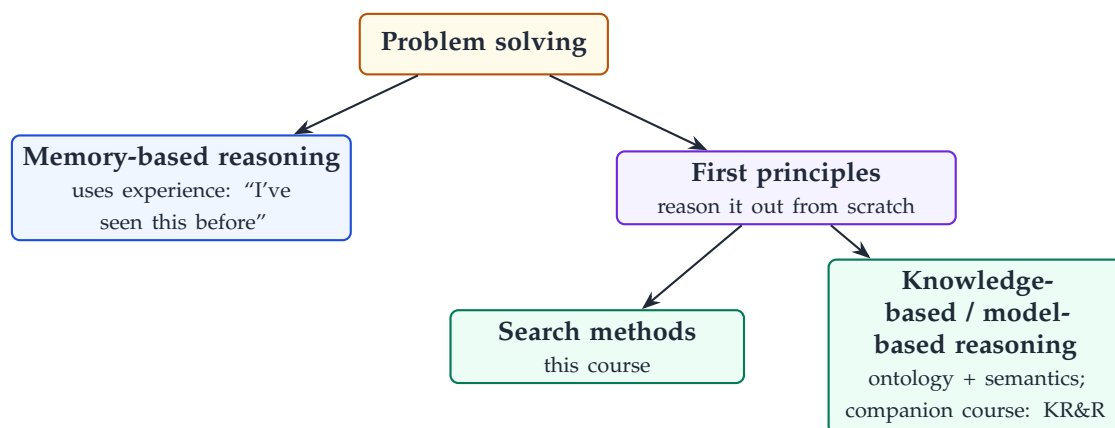
Assumption	Meaning	Relaxed later in
World is static	Nothing changes except through our actions	(dynamic worlds: beyond this course)
World is completely known	Full observability; we know the state exactly	(uncertainty: other courses)
Only one agent changes the world	No opponents or collaborators	Game playing (Minimax, Alpha-Beta)
Actions never fail	Deterministic transitions $T(s, a)$	(probabilistic planning, MDPs)
Representation is taken care of	Someone has chosen states and actions for us	KR & R companion course

Intuition: Why these assumptions are not cheating

Under these assumptions the agent can *plan the whole solution in its head* and then execute it blindly: since the world is static, known and deterministic, the plan will work. That makes search a clean mathematical object. The football example violates every assumption; we will get there gradually.

9.5 Two ways to solve problems

Humankind is a problem-solving species. The slide splits problem solving into two approaches:



9.5.1 Search vs knowledge: the Rubik's cube

Erno Rubik, a Hungarian architect, invented the cube in 1974; reportedly it took him about a month to find his first solution method. Today anyone can look up a solving algorithm and even children solve it in minutes. That is the **knowledge-based** approach: we already know how. Without that knowledge, we must use a **first-principles search** method. More recently, deep reinforcement learning (DeepCubeA, 2018/19) learned to solve the cube efficiently without human guidance — again by combining a learned heuristic with search.

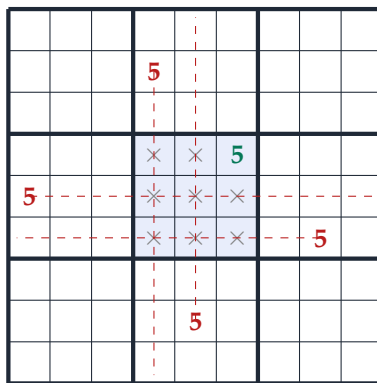
Formula / Derivation: How big is the Rubik's cube state space?

A $3 \times 3 \times 3$ cube has 8 corner pieces and 12 edge pieces (centres are fixed).

- Corner positions: $8!$ permutations. Corner orientations: each of 8 corners has 3 twists, but the last is determined by the other seven (total twist $\equiv 0 \pmod{3}$): 3^7 .
- Edge positions: $12!$. Edge orientations: 2 flips each, last determined (total flip even): 2^{11} .
- Parity: corner and edge permutations must have the same parity, dividing by 2.

$$|S| = \frac{8! \cdot 3^7 \cdot 12! \cdot 2^{11}}{2} = \frac{40320 \cdot 2187 \cdot 479001600 \cdot 2048}{2} \\ = 43,252,003,274,489,856,000 \approx 4.3 \times 10^{19}.$$

Every state can be solved in at most **20 moves** ("God's number", proved in 2010 by exhaustive computer search using symmetry). With 18 possible face turns per move, a naive depth-20 tree has $18^{20} \approx 1.3 \times 10^{25}$ leaves — far more than the number of states, because many move sequences lead to the same state. Recognising repeated states is a key theme of Week 2.

9.5.2 Search vs reasoning: Sudoku

Shaded box needs a 5.

Two rows already contain a 5 (red, horizontal) $\Rightarrow$ 6 cells ruled out.

Two columns already contain a 5 (red, vertical) $\Rightarrow$ 2 more ruled out.

Only one empty cell remains $\Rightarrow$ **5 must go there** (green).

No guessing: this is **reasoning**.

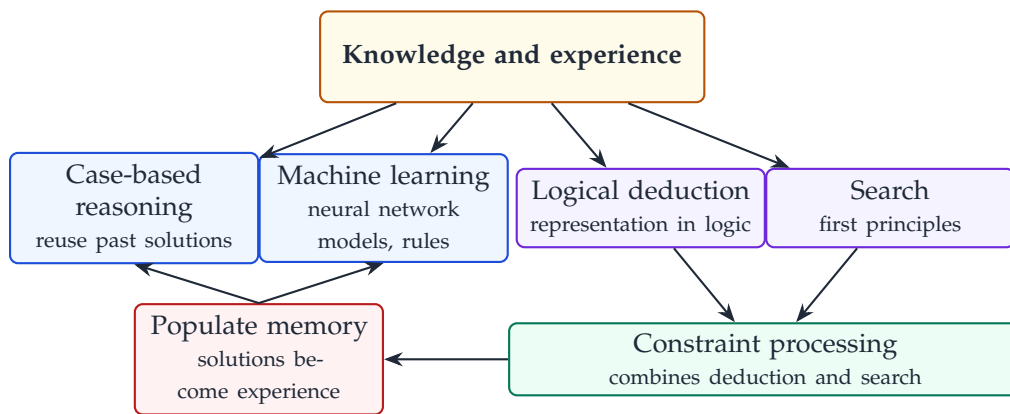
Intuition: Reasoning vs search in Sudoku

- **Reasoning:** deduce forced values from constraints (as above). Fast, no mistakes, but may get stuck on hard puzzles.
- **Search:** when no cell is forced, *guess* a value and continue; if a contradiction appears, *backtrack*.

The slide's key sentence: **both reasoning and search are subsumed by constraint satisfaction**, as the final part of the course shows. Real solvers interleave them: deduce as much as possible, then guess, then deduce again.

9.6 An architecture for problem solving

The slide puts all the pieces together into one architecture:



Intuition: How the loop works

When faced with a new problem, an agent first checks memory (have I solved something like this?) and learned models. If that fails, it falls back on first principles: logical deduction and search, unified in constraint processing. Whatever it solves goes back into memory, so next time it is a memory problem. **Search is the fallback that turns new problems into experience.** A Rubik's solver first solves it by search; later, by memory.

9.7 A map colouring problem

The last slide introduces the running example for constraint satisfaction.

Definition: Map colouring

Colour each region with one of its *allowed* colours such that **no two adjacent regions have the same colour**.

The slide gives five regions with allowed colours (r = red, b = blue, g = green):

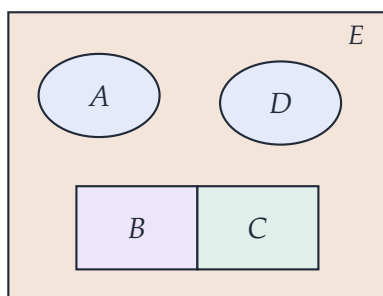
$$A \in \{r, b, g\}, \quad B \in \{b, g\}, \quad C \in \{r, b, g\}, \quad D \in \{b, g\}, \quad E \in \{r, g\},$$

and a **constraint graph** with five " $\neq$ " edges.

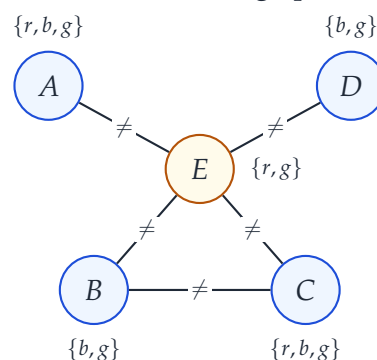
Exam trap: About the adjacencies used here

The slide's picture cannot be fully recovered from its text, so **both the assignment of colour sets to regions and the adjacencies below are a reconstruction**. The five colour sets themselves ($\{r, b, g\}$ twice, $\{b, g\}$ twice, $\{r, g\}$ once) and the count of five $\neq$ edges are from the slide. These notes use the adjacency **E borders every other region, and B borders C** (five $\neq$ edges: $A-E$, $B-E$, $C-E$, $D-E$, $B-C$), which matches the five edges and layout on the slide. If your slide's adjacencies differ, re-run the same method; nothing in the technique changes.

The map



The constraint graph



Definition: Constraint graph

One node per variable (region); an edge between two nodes when a constraint relates them (here, “must differ”). The task becomes: **choose a label (colour) for each node** from its domain such that every edge constraint holds.

Formula / Derivation: Size of the search space

Number of complete assignments = product of domain sizes:

$$|A| \cdot |B| \cdot |C| \cdot |D| \cdot |E| = 3 \cdot 2 \cdot 3 \cdot 2 \cdot 2 = 72.$$

Brute force (generate-and-test) checks all 72. Counting the valid ones: fix E .

- $E = r$: $A \in \{b, g\}$ (2 choices), $D \in \{b, g\}$ (2), $B \in \{b, g\}$, $C \in \{b, g\}$ with $B \neq C$: (b, g) or (g, b) (2). Total $2 \cdot 2 \cdot 2 = 8$.
- $E = g$: $A \in \{r, b\}$ (2), $D \in \{b\}$ (1), $B \in \{b\}$ (1), $C \in \{r, b\} \setminus \{B\} = \{r\}$ (1). Total 2.

So there are $8 + 2 = 10$ solutions out of 72 assignments (verified by computer enumeration).

9.7.1 Solving by backtracking (preview)

Assign variables one at a time; after each assignment check the constraints among assigned variables; on a violation, try the next value; if no value works, **backtrack** to the previous variable.

Trace 1: order A, B, C, D, E ; values tried in order g, r, b .

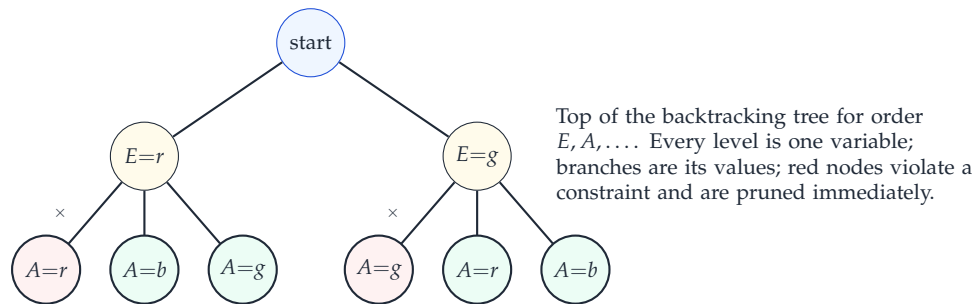
Step	Action	Result
1	$A = g$	ok
2	$B = g$	ok (A, B not adjacent)
3	$C = g$	fails: $C = B$
4	$C = r$	ok
5	$D = g$	ok
6	$E = g$	fails: $E = A$
7	$E = r$	fails: $E = C$
8		backtrack to D
9	$D = b$	ok
10–11	$E = g, E = r$	both fail (same reasons)
12–13		backtrack past D to C
14	$C = b$	ok
15	$D = g$	ok
16	$E = g$	fails: $E = A$
17	$E = r$	ok — solution $A=g, B=g, C=b, D=g, E=r$

Trace 2: order E, A, B, C, D (start with E , the node with the *most* constraints); same value order. $E = g$; $A = g$ fails, $A = r$ ok; $B = g$ fails, $B = b$ ok; $C = g$ fails, $C = r$ ok; $D = g$ fails, $D = b$ ok. Solution $E=g, A=r, B=b, C=r, D=b$ with **no backtracking**.

Intuition: The lesson of the two traces

In trace 1, the real problem was E , which touches everything, but it was assigned last, so we discovered the conflict late and had to undo good work. Assigning the most constrained / most constraining variable first finds conflicts early. **Variable ordering is a heuristic**, and heuristics are what make search efficient. You will also see *forward checking* and *arc consistency*, which

prune domains after each assignment (e.g. once $E = g$ is chosen, B and D lose g immediately, leaving $\{b\}$), so doomed choices are never even tried.



Module recap

- Problem solving: an autonomous agent in a world, with a goal and a set of actions. Formally $\langle S, s_0, A, T, G \rangle$; a solution is an action sequence reaching G .
- The real world has other events and other agents; we start with static, known, single-agent, deterministic worlds with a given representation.
- Problem solving via memory/experience or via first principles (search = this course; knowledge-based reasoning = KR&R).
- Rubik's cube: $\approx 4.3 \times 10^{19}$ states; knowledge (known algorithms) vs search. Sudoku: reasoning forces values; search guesses and backtracks; constraint satisfaction subsumes both.
- Architecture: knowledge & experience $\rightarrow$ CBR, ML, deduction, search $\rightarrow$ constraint processing $\rightarrow$ populate memory.
- Map colouring as a CSP: 72 assignments, 10 solutions; backtracking; variable ordering matters.

Practice: Test Yourself

Work each problem before reading its solution. Problems are tagged with the module they draw on.

Part A — Numerical problems with full solutions

Problem N1

[Module 2] Training a perceptron on AND. Start with $\mathbf{w} = (0,0)$, $b = 0$, $\eta = 1$, output 1 iff $z = \mathbf{w}^\top \mathbf{x} + b \geq 0$. Present the examples in the order $(0,0)$, $(0,1)$, $(1,0)$, $(1,1)$ with targets $0, 0, 0, 1$, repeating epochs until an epoch makes no updates. Give the final weights and verify them.

Solution

Update rule: $\mathbf{w} \leftarrow \mathbf{w} + \eta(y - \hat{y})\mathbf{x}$, $b \leftarrow b + \eta(y - \hat{y})$.

Epoch	$\mathbf{x}$	y	z	$\hat{y}$	after update ($\mathbf{w}; b$)
1	(0,0)	0	0	1	(0,0); -1
	(0,1)	0	-1	0	no change
	(1,0)	0	-1	0	no change
	(1,1)	1	-1	0	(1,1); 0
2	(0,0)	0	0	1	(1,1); -1
	(0,1)	0	0	1	(1,0); -2
	(1,0)	0	-1	0	no change
	(1,1)	1	-1	0	(2,1); -1
3	(0,0)	0	-1	0	no change
	(0,1)	0	0	1	(2,0); -2
	(1,0)	0	0	1	(1,0); -3
	(1,1)	1	-2	0	(2,1); -2
4	(0,0),(0,1)	0	-2, -1	0	no change
	(1,0)	0	0	1	(1,1); -3
	(1,1)	1	-1	0	(2,2); -2
5	(0,0)	0	-2	0	no change
	(0,1)	0	0	1	(2,1); -3
	(1,0),(1,1)	0,1	-1,0	0,1	no change
6	all four		-3, -2, -1, 0	0,0,0,1	no updates: converged

Final: $\mathbf{w} = (2,1)$, $b = -3$. Check: $(0,0) \rightarrow -3$, $(0,1) \rightarrow -2$, $(1,0) \rightarrow -1$ (all < 0 , output 0); $(1,1) \rightarrow 0$ (≥ 0 , output 1). Correct AND. Convergence is guaranteed because AND is linearly separable.

Problem N2

[Module 2] Verifying an XOR network. Hidden units: $h_1 = \text{step}(x_1 + x_2 - 0.5)$, $h_2 = \text{step}(x_1 + x_2 - 1.5)$. Output: $y = \text{step}(h_1 - 2h_2 - 0.5)$ ($\text{step}(z) = 1$ iff $z \geq 0$). Show that $y = x_1 \oplus x_2$.

Solution

x_1	x_2	$x_1+x_2-0.5$	h_1	$x_1+x_2-1.5$	h_2	$h_1 - 2h_2 - 0.5 \rightarrow y$
0	0	-0.5	0	-1.5	0	-0.5 $\rightarrow$ 0
0	1	0.5	1	-0.5	0	0.5 $\rightarrow$ 1
1	0	0.5	1	-0.5	0	0.5 $\rightarrow$ 1
1	1	1.5	1	0.5	1	-1.5 $\rightarrow$ 0

The output matches XOR. h_1 is OR, h_2 is AND, and the output computes “OR and not AND”.

Problem N3

[Module 2] One step of backpropagation. A 1–1–1 network: $a_1 = \sigma(w_1x)$, $a_2 = \sigma(w_2a_1)$ (no biases), loss $L = \frac{1}{2}(y - a_2)^2$. With $x = 1$, $y = 1$, $w_1 = 0.5$, $w_2 = -1$, $\eta = 0.5$: compute the forward pass, both gradients, the updated weights, and show that the loss decreases.

Solution

Forward: $z_1 = 0.5$, $a_1 = \sigma(0.5) = 0.6225$; $z_2 = -0.6225$, $a_2 = \sigma(-0.6225) = 0.3492$. $L = \frac{1}{2}(1 - 0.3492)^2 = 0.2118$.

Backward (chain rule, using $\sigma' = \sigma(1 - \sigma)$):

$$\delta_2 = \frac{\partial L}{\partial z_2} = (a_2 - y) a_2(1 - a_2) = (-0.6508)(0.3492)(0.6508) = -0.1479$$

$$\frac{\partial L}{\partial w_2} = \delta_2 a_1 = -0.1479 \times 0.6225 = -0.0921$$

$$\delta_1 = \delta_2 w_2 a_1(1 - a_1) = (-0.1479)(-1)(0.6225)(0.3775) = 0.0348$$

$$\frac{\partial L}{\partial w_1} = \delta_1 x = 0.0348$$

Update: $w_2 \leftarrow -1 - 0.5(-0.0921) = -0.9540$; $w_1 \leftarrow 0.5 - 0.5(0.0348) = 0.4826$.

New forward: $a_1 = \sigma(0.4826) = 0.6184$, $a_2 = \sigma(-0.9540 \times 0.6184) = \sigma(-0.5899) = 0.3567$, $L = \frac{1}{2}(0.6433)^2 = 0.2069 < 0.2118$. ✓ Loss decreased.

Interpretation: w_2 was negative and pushing a_2 down; the gradient step makes it less negative.

Problem N4

[Module 2] Softmax and cross-entropy. A 3-class classifier (Horse, Cow, Lion) outputs logits $\mathbf{z} = (2, 1, 0)$. The true class is Cow. Compute the probabilities, the loss, and $\partial L / \partial \mathbf{z}$.

Solution

$e^2 = 7.389$, $e^1 = 2.718$, $e^0 = 1$; sum = 11.107.

$$\hat{\mathbf{y}} = (0.665, 0.245, 0.090).$$

Loss $L = -\log \hat{y}_{\text{Cow}} = -\ln 0.245 = 1.408$.

Gradient $\hat{\mathbf{y}} - \mathbf{y} = (0.665, 0.245 - 1, 0.090) = (0.665, -0.755, 0.090)$ [3×1].

Gradient descent subtracts this, so it raises the Cow logit and lowers the others, as it should.

Problem N5

[Module 2] Parameter count. An MLP classifies 28×28 grey-scale images into 10 digits with one hidden layer of 128 units. How many trainable parameters? Give the shape of each weight matrix.

Solution

$d = 784, h = 128, K = 10$. $W^{(1)} \in \mathbb{R}^{128 \times 784}$: 100,352; $\mathbf{b}^{(1)}$: 128; $W^{(2)} \in \mathbb{R}^{10 \times 128}$: 1,280; $\mathbf{b}^{(2)}$: 10.
Total = $100,352 + 128 + 1,280 + 10 = \mathbf{101,770}$.

Problem N6

[Module 6] Babbage's method of differences. For $p(x) = 2x^3 - x + 5$: (a) build the difference table for $x = 0, \dots, 6$; (b) predict the constant third difference from theory; (c) compute $p(7)$ using additions only and check it.

Solution

(a)

x	0	1	2	3	4	5	6
p	5	6	19	56	129	250	431
Δ	1	13	37	73	121	181	
Δ^2	12	24	36	48	60		
Δ^3	12	12	12	12			

(b) Theory: $\Delta^n p = n! a_n = 3! \times 2 = 12$. ✓

(c) Cascade: $\Delta_{\text{new}}^2 = 60 + 12 = 72$; $\Delta_{\text{new}} = 181 + 72 = 253$; $p(7) = 431 + 253 = \mathbf{684}$.
Check: $2(343) - 7 + 5 = 684$. ✓

Problem N7

[Module 6] Pascaline subtraction. Compute $7042 - 2589$ using only addition and nines' complement, as a Pascaline would.

Solution

Nines' complement of 2589 (4 digits): $9999 - 2589 = 7410$ (digit-wise $9 - d$). $7042 + 7410 = 14452$. Add 1: 14453. Drop the leading 1 (subtract 10^4): **4453**.

Check: $2589 + 4453 = 7042$. ✓

Why it works: $a + (10^n - 1 - b) + 1 - 10^n = a - b$.

Problem N8

[Module 6] Stepped Reckoner efficiency. How many additions does the shift-and-add method need for 1234×509 , compared with naive repeated addition? Compute the product this way.

Solution

Digits of 509: $5, 0, 9 \Rightarrow 9 + 0 + 5 = \mathbf{14}$ additions (versus 509 naively, a $36\times$ saving).

Units: $9 \times 1234 = 11106$. Tens: $0 \times 12340 = 0$. Hundreds: $5 \times 123400 = 617000$.

Total: $11106 + 0 + 617000 = \mathbf{628106}$. Check: $1234 \times 509 = 628106$. ✓

Problem N9

[Module 4] Cost of blind search vs knowledge. A problem has branching factor $b = 3$ and solution depth $d = 10$. (a) How many nodes does a full blind search to depth 10 generate (counting the root)? (b) A heuristic gives an effective branching factor $b^* = 1.5$. How many nodes now? (c) What does a perfect heuristic need?

Solution

$$(a) \sum_{i=0}^{10} 3^i = \frac{3^{11} - 1}{3 - 1} = \frac{177147 - 1}{2} = 88,573.$$

$$(b) \sum_{i=0}^{10} 1.5^i = \frac{1.5^{11} - 1}{0.5} \approx \frac{86.50 - 1}{0.5} \approx 171 \text{ — about 500 times fewer.}$$

(c) It picks the right child each time: it generates b children at each of d levels, $3 \times 10 = 30$ nodes (31 with the root).

This is Elaine Rich's definition in numbers: knowledge turns exponential work into (near) linear work.

Problem N10

[Module 4] Winograd schemas and guessing. Each Winograd question has 2 choices. (a) A system that guesses answers 20 questions. What is the probability it gets at least 15 right? (b) Same for at least 36 of 40. (c) What do these numbers say about using accuracy as evidence of understanding?

Solution

$$X \sim \text{Bin}(n, \frac{1}{2}), P(X \geq k) = 2^{-n} \sum_{j \geq k} \binom{n}{j}.$$

$$(a) \binom{20}{15} + \binom{20}{16} + \cdots + \binom{20}{20} = 15504 + 4845 + 1140 + 190 + 20 + 1 = 21700; 21700/1048576 \approx 0.0207 \text{ (about 2\%).}$$

$$(b) \sum_{j=36}^{40} \binom{40}{j} = 91390 + 9880 + 780 + 40 + 1 = 102091; /2^{40} \approx 9.3 \times 10^{-8}.$$

(c) Binary questions make chance performance 50%, so a meaningful test needs many questions: 75% on 20 questions happens by luck 1 time in 50, but 90% on 40 essentially never does. Even then, high accuracy shows the *performance*; whether it reflects *competence* (Module 2) is a separate question.

Problem N11

[Module 8] Atoms in a person. Estimate the number of atoms in a 50 kg person, modelling the body as water (18 g/mol, 3 atoms per molecule). Show units.

Solution

$$\text{Moles: } 50000 \text{ g} / 18 \text{ g mol}^{-1} = 2778 \text{ mol.}$$

$$\text{Molecules: } 2778 \times 6.022 \times 10^{23} = 1.673 \times 10^{27}.$$

$$\text{Atoms: } \times 3 = 5.0 \times 10^{27}, \text{ consistent with the slide's "about } 10^{27} \text{ " to within an order of magnitude.}$$

Problem N12

[Module 8] Orders of magnitude. Using the slide values (Earth diameter 12,756 km, Earth–Sun distance 149,600,000 km): (a) how many Earth diameters fit between Earth and Sun? (b) How many orders of magnitude separate a virus (10^{-7} m) from the Milky Way (10^{21} m)?

Solution

- (a) $1.496 \times 10^8 / 1.2756 \times 10^4 \approx \mathbf{11,728}$ Earth diameters ($\approx 10^4$).
 (b) $10^{21} / 10^{-7} = 10^{28}$: **28 orders of magnitude**.

Problem N13

[Module 9] Rubik's cube: states vs move sequences. (a) Verify $|S| = \frac{8! \cdot 3^7 \cdot 12! \cdot 2^{11}}{2}$ numerically. (b) With 18 face turns per move, how many move sequences of length 20 are there? (c) Roughly how many sequences per state? What does this imply for search?

Solution

- (a) $8! = 40320$, $3^7 = 2187$, $12! = 479001600$, $2^{11} = 2048$. $40320 \times 2187 = 88,179,840$; $\times 479001600 = 4.2238 \times 10^{16}$; $\times 2048 = 8.6504 \times 10^{19}$; $/2 = \mathbf{4.3252 \times 10^{19}}$ (exactly 43,252,003,274,489,856,000).
 (b) $18^{20} = 1.275 \times 10^{25}$.
 (c) $1.275 \times 10^{25} / 4.325 \times 10^{19} \approx \mathbf{2.9 \times 10^5}$ sequences per state on average. A tree search that does not detect repeated states wastes enormous work revisiting the same configurations; a *graph* search that remembers visited states avoids this (Week 2).

Problem N14

[Module 9] Map colouring variants. Using the notes' map ($A\{r, b, g\}$, $B\{b, g\}$, $C\{r, b, g\}$, $D\{b, g\}$, $E\{r, g\}$; edges $A-E$, $B-E$, $C-E$, $D-E$, $B-C$): (a) How many solutions if the border between B and C is removed? (b) How many if E must be red? (c) If E must be green, which variables have their colour forced?

Solution

- (a) Fix E . $E = r$: $A \in \{b, g\}$, $B \in \{b, g\}$, $C \in \{b, g\}$, $D \in \{b, g\}$, all free: $2^4 = 16$. $E = g$: $A \in \{r, b\}$ (2), $B = b$ (1), $C \in \{r, b\}$ (2), $D = b$ (1): 4. Total **20**.
 (b) From the notes' count, $E = r$ gives **8** solutions.
 (c) $E = g$ removes g from B and D , forcing $B = b$, $D = b$; then $C \neq E$, $C \neq B$ forces $C = r$. Only $A \in \{r, b\}$ remains free (2 solutions). This propagation of forced values is exactly what *arc consistency* automates.

Problem N15

[Module 6] ENIAC economics. ENIAC drew 150 kW. (a) Energy used in a 24-hour day? (b) At Rs. 8 per kWh, what is the daily electricity cost? (c) If it did 5,000 additions per second, how many additions per joule?

Solution

- (a) $150 \text{ kW} \times 24 \text{ h} = \mathbf{3600 \text{ kWh}}$.
 (b) $3600 \times 8 = \mathbf{\text{Rs. } 28,800}$ per day.
 (c) $150 \text{ kW} = 1.5 \times 10^5 \text{ J/s}$; $5000 / 1.5 \times 10^5 \approx \mathbf{0.033}$ additions per joule (about 30 J per addition). Modern chips perform on the order of 10^{11} – 10^{12} operations per joule.

Problem N16

[Module 6] Lull's combinatorics. A Lullian disc has 16 concepts. (a) How many unordered pairs of distinct concepts can be formed? (b) Three concentric discs of 16 symbols each: how

many alignments? (c) If a human needs 1 minute to evaluate each alignment, how long to examine them all?

Solution

(a) $\binom{16}{2} = \frac{16 \cdot 15}{2} = 120$.

(b) $16^3 = 4096$.

(c) 4096 min $\approx$ 68 hours, almost 3 days of non-stop work. Even medieval generate-and-test ran into combinatorial explosion: the generator is cheap, the tester is expensive. Heuristics prune what the tester has to look at.

Problem N17

[Module 5] Galileo's law of falling bodies. (a) Using $s = \frac{1}{2}gt^2$ with $g = 9.8 \text{ m s}^{-2}$, how far does a stone fall in the 1st, 2nd and 3rd seconds? (b) Show that these distances are in ratio 1 : 3 : 5.

Solution

(a) $s(1) = 4.9 \text{ m}$, $s(2) = 19.6 \text{ m}$, $s(3) = 44.1 \text{ m}$. Distances in each second: 4.9 , $19.6 - 4.9 = 14.7$, $44.1 - 19.6 = 24.5 \text{ m}$.

(b) $4.9 : 14.7 : 24.5 = 1 : 3 : 5$. Generally $s(n) - s(n-1) = \frac{1}{2}g(2n-1)$, the odd numbers. Galileo reached this through geometry (areas of trapezia in a velocity–time diagram) — the representation of motion by figures that Hobbes took as the model for thought.

Part B — Subjective questions with model answers

Question S1

List the four properties of an intelligent agent from the lecture and explain why a world model is central to problem solving.

Solution

An agent is **persistent** (exists over time with a history), **autonomous** (chooses its own actions), **proactive** (takes initiative rather than only reacting) and **goal-directed** (behaviour organised around goals). It carries a model of the world, possibly abstract; a self-aware agent includes itself. The model matters because problem solving by search means *simulating* action sequences before executing them. Without an internal model, an agent could only try actions in the real world and live with the consequences. The model lets it explore alternatives cheaply and safely, and pick one that reaches the goal.

Question S2

Explain the “signal $\rightarrow$ symbol $\rightarrow$ signal” information-processing view and where machine learning and search sit in it.

Solution

Agents receive continuous, noisy signals (images, sound) and must output continuous signals (motor commands). In between, deliberation works best on discrete symbols (“ball”, “defender left”). Perception converts signal to symbol; motor control converts symbol back to signal; symbolic reasoning connects them. Neuro-fuzzy methods and deep learning dominate

the conversion stages (especially perception); search, planning and logical inference dominate the symbolic middle. The cycle is sense–deliberate–act, looping through the world.

Question S3

Why could a single-layer perceptron not learn XOR, and how did multi-layer networks overcome this? Why was the answer important historically?

Solution

A perceptron outputs $\text{step}(\mathbf{w}^\top \mathbf{x} + b)$, whose decision boundary is a single hyperplane. XOR's positive points $(0, 1)$, $(1, 0)$ and negative points $(0, 0)$, $(1, 1)$ lie on opposite diagonals, so no line separates them; the four inequalities $\theta > 0$, $w_1 \geq \theta$, $w_2 \geq \theta$, $w_1 + w_2 < \theta$ are contradictory. A hidden layer lets each unit draw a line and the output combine the half-planes (OR and not AND). Backpropagation made such networks trainable by passing error gradients backwards through hidden layers. Historically, the linear-separability limitation dampened neural network research for years; the 1986 backprop work revived it, and deep networks with many layers (2012) launched the current ML era.

Question S4

Distinguish performance from competence using the Frisbee example, and relate it to suitcase words.

Solution

Performance is succeeding at a specific task; competence is the general understanding we presume lies behind it. A person who describes a photo as “people playing Frisbee in a park” can also answer questions about Frisbee shapes, throwing distances, edibility and parks, because humans' performance rests on broad competence. An image captioner can produce the same caption with no knowledge of persons, parks or physics. We err when we infer competence from machine performance. Suitcase words like “learning” encourage this error: “the machine learned X” evokes the rich human sense of learning, when the machine's learning is narrow, brittle and heavily engineered for one domain.

Question S5

Compare the Turing test with the Winograd Schema Challenge. What weakness of the former does the latter try to fix?

Solution

The Turing test judges a machine by whether a human can distinguish it from a person in free conversation. It is behavioural and avoids defining “think”, but it rewards deception: chatbots like ELIZA and Izar fool judges with deflection, personality and topic changes, without understanding. The Winograd Schema Challenge replaces open conversation with binary pronoun-resolution questions built so that one word (“big”/“small”, “feared”/“advocated”) flips the answer while leaving word statistics nearly unchanged. Both candidate nouns are in the same semantic class, so grammar gives nothing away. The design aims to be Google-proof and to require commonsense knowledge, and its format (fixed answers) removes the judge and the opportunity for trickery.

Question S6

Take the sentence “The trophy would not fit in the brown suitcase because it was too big.” Identify each component of the Winograd schema and explain what knowledge resolves it.

Solution

Noun phrases of the same class (inanimate objects): *the trophy, the suitcase*. Ambiguous pronoun: *it*. Special word: *big*; alternate word: *small*. Question: what was too big? Answers: trophy / suitcase. Resolution needs spatial commonsense: when X does not fit inside Y, it is because X is too large or Y too small. So with “big”, *it* = trophy; with “small”, *it* = suitcase.

Question S7

Trace the idea that “thinking is symbol manipulation” from Galileo to Newell and Simon.

Solution

Galileo: perceived qualities (colour, taste) exist in the perceiver, and geometry can represent and reason about motion. **Hobbes**: extended this to thought — reasoning is computation, adding and subtracting mental symbols. **Descartes**: geometry can be turned into algebra; thoughts are symbolic representations; but mind and world are distinct (dualism), raising the question of how symbols get meaning and how mind and matter interact. **Leibniz**: dreamed of a calculus of reasoning that would settle disputes by calculation. **Lovelace**: saw that a computing engine could manipulate any symbols, not just numbers. **Turing**: formalised computation and proposed a behavioural test. **Newell & Simon**: built the Logic Theorist and GPS and stated the Physical Symbol System Hypothesis — a physical symbol system has the necessary and sufficient means for general intelligent action.

Question S8

State and explain the paradox of mechanical reason. How does AI claim to resolve it?

Solution

If reasoning is manipulating *meaningful* symbols by rational rules, who does the manipulating? A mechanism follows rules blindly and cannot attend to meaning; something that attends to meaning seems to need a mind of its own — a homunculus — which leads to infinite regress. So reasoning seems to be either mechanical or meaningful, but not both. AI’s answer is that in a well-designed formal system, purely syntactic operations are *meaning-preserving*: carry-based addition or algebraic manipulation proceeds mechanically, yet the results stay true of what the symbols represent. A computer is exactly such a mechanism. Critics (Searle) reply that preserving meaning for *us* is not the same as the machine *having* meaning.

Question S9

State the Physical Symbol System Hypothesis. Separate its two claims and name a critic of each.

Solution

“A physical symbol system has the necessary and sufficient means for general intelligent action” (Newell & Simon). **Sufficiency**: a suitably organised symbol system can be intelligent, so computers can in principle think. Searle’s Chinese Room attacks this: rule-following on symbols is not understanding. **Necessity**: anything intelligent must be a symbol system,

so humans are symbol processors. Dreyfus attacks this: intelligence rests on unconscious skills and instincts that cannot be captured as formal rules; connectionists also argue much intelligence is sub-symbolic. The PSSH defines symbolic AI, which Haugeland called GOFAL.

Question S10

Why do we not represent the world in terms of fundamental particles, and what principle replaces this?

Solution

In principle the laws of particle physics explain everything, but a single human contains around 10^{27} atoms interacting with countless others. Writing and solving the equations is infeasible (simulating one second would need around 10^{42} operations), and even if solved, a list of particle positions would not be meaningful to us. Instead each domain of study picks an appropriate grain — atoms for chemistry, cells for biology, people for sociology — and **every domain defines its own ontology**. For AI, choosing the right ontology is the first step in formulating a problem: it determines the size of the state space and whether search is feasible.

Question S11

List the simplifying assumptions with which the course begins and state where each is relaxed.

Solution

(1) The world is static — nothing changes except by our actions. (2) The world is completely known. (3) Only one agent changes the world — relaxed in game playing (Minimax, Alpha-Beta, SSS*). (4) Actions never fail (deterministic). (5) Representation is given. Under these assumptions, a plan found by search in the model is guaranteed to work when executed. Uncertainty, dynamism and learning representations are handled in other courses; knowledge representation in the companion KR&R course.

Question S12

Using the Rubik's cube and Sudoku, explain the difference between knowledge-based, search-based and reasoning-based problem solving.

Solution

Knowledge: today anyone can learn a cube algorithm and solve it in minutes; the solution method is already known and simply applied. **Search:** without that knowledge, one must explore move sequences from first principles — Rubik took about a month; a computer faces $\approx 4.3 \times 10^{19}$ states. **Reasoning:** in Sudoku, constraints can *force* a value (the 5 must go in the only remaining cell), with no guessing. When reasoning gets stuck, search (guess and backtrack) takes over. Constraint satisfaction subsumes both: propagate constraints as far as possible, then branch.

Question S13

Describe means–ends analysis and one weakness.

Solution

Means–ends analysis (GPS) repeatedly finds the most important difference between the current state and the goal, selects an operator that reduces it, and, if the operator’s preconditions do not hold, sets up achieving them as a subgoal and recurses. It mirrors human planning (handle the big gap — fly — then the details — cab, walk). Weakness: it greedily reduces differences, so it fails on problems where you must temporarily move *away* from the goal (e.g. moving small discs aside in the Tower of Hanoi, or detours in route finding).

Question S14

Describe the lecture’s architecture for problem solving and explain the role of search in it.

Solution

Knowledge and experience feed four methods: case-based reasoning (reuse similar past solutions), machine learning (neural network models, learned rules), logical deduction (from knowledge represented in logic) and search (first principles). Deduction and search combine in constraint processing. Solutions found are used to populate memory, which feeds case-based reasoning and learning next time. Search is the general fallback for novel problems: when memory and learned models do not apply, search can still find an answer, and that answer becomes new experience.

Part C — Multiple-choice questions

Answers and explanations follow the questions.

- Q1.** Which is *not* one of the four agent properties shown in the lecture?
 (a) Persistent (b) Proactive (c) Omniscient (d) Goal directed
- Q2.** In the information-processing view, symbolic reasoning operates on:
 (a) raw signals (b) symbols (c) motor commands (d) pixel intensities
- Q3.** A single-layer perceptron can represent:
 (a) XOR (b) any Boolean function (c) only linearly separable functions (d) no function with two inputs
- Q4.** The derivative of the sigmoid is:
 (a) $\sigma(z)^2$ (b) $\sigma(z)(1 - \sigma(z))$ (c) $1 - \sigma(z)^2$ (d) e^{-z}
- Q5.** In backprop with $W^{(2)} \in \mathbb{R}^{K \times h}$, $\delta^{(2)} \in \mathbb{R}^K$, the product $W^{(2)\top} \delta^{(2)}$ has shape:
 (a) $K \times 1$ (b) $h \times 1$ (c) $h \times K$ (d) $K \times h$
- Q6.** According to the slides, who won the 2018 Turing Award?
 (a) Minsky, McCarthy, Simon (b) Hinton, LeCun, Bengio (c) Rumelhart, Hinton, Williams
 (d) Newell, Simon, Shaw
- Q7.** “Suitcase words” is a term due to:
 (a) Rodney Brooks (b) Marvin Minsky (c) Alan Turing (d) John Searle
- Q8.** Darwiche’s argument is that current ML delivers:
 (a) human-level intelligence (b) animal-like abilities (c) consciousness (d) commonsense reasoning

- Q9.** The course's focus is on which kind of knowledge?
(a) tacit (b) procedural (c) explicit declarative (d) implicit statistical
- Q10.** Ant pheromone trails are the slide's example of:
(a) semantics (b) biosemiotics (c) ontology (d) dualism
- Q11.** Which definition of AI emphasises solving exponentially hard problems in polynomial time using domain knowledge?
(a) Simon (b) Charniak & McDermott (c) Elaine Rich (d) Barr & Feigenbaum
- Q12.** Searle's Chinese Room is an argument against the idea that:
(a) machines can calculate (b) symbol manipulation alone yields understanding (c) humans understand Chinese (d) the Turing test exists
- Q13.** Turing called his test:
(a) the Loebner game (b) the Imitation Game (c) the Winograd challenge (d) the Chinese Room
- Q14.** ELIZA's DOCTOR script imitated a:
(a) Freudian analyst (b) Rogerian psychotherapist (c) general physician (d) chess master
- Q15.** In a Winograd schema, the two noun phrases must:
(a) differ in gender (b) be of the same semantic class (c) be proper names (d) appear in different sentences
- Q16.** "The lawyer asked the witness a question, but he was reluctant to answer it." Who was reluctant?
(a) the lawyer (b) the witness (c) ambiguous (d) neither
- Q17.** "John took the water bottle out of the backpack so that it would be lighter." "It" refers to:
(a) the bottle (b) the backpack (c) John (d) the water
- Q18.** The slides call this philosopher the "grandfather of AI":
(a) Descartes (b) Galileo (c) Hobbes (d) Leibniz
- Q19.** "Geometry $\rightarrow$ algebra" is associated with:
(a) Galileo (b) Descartes (c) Pascal (d) Kant
- Q20.** The paradox of mechanical reason asks:
(a) whether machines can be built from metal (b) how a manipulator can be both mechanical and attentive to meaning (c) whether reason is emotional (d) why computers are fast
- Q21.** Vaucanson's duck and von Kempelen's Turk both illustrate:
(a) genuine machine intelligence (b) appearance of a capability without the real thing (c) steam power (d) punched-card programming
- Q22.** The Pascaline could directly:
(a) multiply and divide (b) add and subtract (c) compute polynomials (d) run programs
- Q23.** Babbage's Difference Engine was designed to compute:
(a) logarithms by table lookup (b) values of polynomial functions (c) chess moves (d) Bernoulli numbers only
- Q24.** For a polynomial of degree 4 with leading coefficient 3, the constant 4th difference is:
(a) 12 (b) 24 (c) 72 (d) 81

- Q25.** Which feature made the Analytical Engine Turing-complete in modern terms?
(a) its weight (b) conditional branching and loops with memory (c) vacuum tubes (d) printing output
- Q26.** Babbage borrowed punched cards from:
(a) the Arithmometer (b) the Jacquard loom (c) the zairja (d) ENIAC
- Q27.** The name “artificial intelligence” is credited to:
(a) Alan Turing (b) John McCarthy (c) Herbert Simon (d) Claude Shannon
- Q28.** The Logic Theorist proved theorems from:
(a) Euclid’s *Elements* (b) Russell & Whitehead’s *Principia Mathematica* (c) Newton’s *Principia* (d) Gödel’s papers
- Q29.** GPS used which strategy?
(a) minimax (b) means–ends analysis (c) backpropagation (d) genetic algorithms
- Q30.** The PSSH states that a physical symbol system is ___ for general intelligent action.
(a) necessary only (b) sufficient only (c) necessary and sufficient (d) irrelevant
- Q31.** Which language did Newell design, in which LT was written?
(a) Lisp (b) IPL (c) Prolog (d) FORTRAN
- Q32.** “Every domain of study defines its own ____.”
(a) algorithm (b) ontology (c) heuristic (d) hypothesis
- Q33.** Which is *not* one of the initial simplifying assumptions?
(a) the world is static (b) actions never fail (c) multiple adversaries act simultaneously (d) the world is completely known
- Q34.** The slides say that both reasoning and search are subsumed by:
(a) machine learning (b) constraint satisfaction (c) case-based reasoning (d) the Turing test
- Q35.** For the notes’ map-colouring problem (domain sizes 3, 2, 3, 2, 2), the number of complete assignments is:
(a) 12 (b) 36 (c) 72 (d) 243
- Q36.** In backtracking for map colouring, assigning the most-connected variable first mainly:
(a) guarantees optimality (b) detects conflicts earlier, reducing backtracking (c) increases the number of solutions (d) removes constraints
- Q37.** AlphaGo combined deep networks with:
(a) ELIZA-style patterns (b) Monte Carlo tree search (c) the difference engine (d) means–ends analysis only
- Q38.** Blind search to depth d with branching factor b generates $\Theta(\text{---})$ nodes:
(a) bd (b) d^b (c) b^d (d) $\log_b d$
- Q39.** Ada Lovelace’s key insight beyond number crunching was that the Engine could:
(a) run on steam (b) manipulate any symbols whose relations could be expressed, e.g. compose music (c) replace humans entirely (d) weave cloth
- Q40.** Leibniz’s word *calculemus* expresses the hope that:
(a) disputes can be settled by calculation (b) calculators will be cheap (c) calculus will replace geometry (d) machines will feel

Answers and explanations

Q	Ans	Explanation
1	c	The four are persistent, autonomous, proactive, goal directed. Omniscience is not required; the model may be an abstraction.
2	b	Signals are handled by perception and motor control; reasoning is in the symbolic middle.
3	c	One hyperplane $\Rightarrow$ only linearly separable functions; XOR is the classic counter-example.
4	b	Derived in Module 2: $\sigma' = \sigma(1 - \sigma)$, max 0.25 at $z = 0$.
5	b	$(h \times K)(K \times 1) = h \times 1$ — one error value per hidden unit.
6	b	The slides name Hinton, LeCun and Bengio.
7	b	Minsky coined it; Brooks used it in his essay on AI predictions.
8	b	“Animal-like abilities”: eagles see better, dogs react to speech, but neither reasons like humans.
9	c	The slide explicitly excludes implicit procedural and tacit knowledge.
10	b	Biosemiotics: complex behaviour from simple systems interacting through signs.
11	c	Rich’s definition links exponential hardness and domain knowledge — the spirit of heuristic search.
12	b	Syntax (rules) is not semantics (understanding), argues Searle.
13	b	The Imitation Game, from the 1950 paper in <i>Mind</i> .
14	b	Rogsonian therapy reflects the patient’s words back, which suits pattern-based rewriting.
15	b	Same class so grammatical agreement cannot resolve the pronoun.
16	b	The one asked a question is the one who would answer it.
17	b	Removing the bottle makes the <i>backpack</i> lighter. With “handy”, it would be the bottle.
18	c	Hobbes first proposed that thinking is symbol manipulation and reasoning is computation.
19	b	Galileo: motion $\rightarrow$ geometry; Descartes: geometry $\rightarrow$ algebra.
20	b	Mechanical or meaningful — how can it be both?
21	b	The duck faked digestion; the Turk hid a human chess master.
22	b	It added and subtracted directly; multiplication and division were by repetition.
23	b	Polynomials via finite differences, needing only additions. (Bernoulli numbers were Lovelace’s program for the Analytical Engine.)
24	c	$\Delta^n p = n! a_n = 4! \times 3 = 72$.
25	b	Branching and loops plus a store (memory) make it general-purpose.
26	b	Jacquard’s loom used chains of punched cards to select weaving patterns.
27	b	McCarthy, for the 1956 Dartmouth proposal.
28	b	LT found shorter proofs for some <i>Principia Mathematica</i> theorems.
29	b	Reduce the biggest difference; recurse on unmet preconditions.
30	c	Both necessary and sufficient — two separate claims.
31	b	IPL (Information Processing Language). Lisp was McCarthy’s.
32	b	The ontology fixes what things exist at that level of description.
33	c	Only one agent changes the world initially; adversaries come with game playing.
34	b	Stated on the Sudoku slide; the final syllabus unit is constraint processing.
35	c	$3 \times 2 \times 3 \times 2 \times 2 = 72$.
36	b	Variable ordering is a heuristic: find failures early. It does not change the solution set.

Q	Ans	Explanation
37	b	Policy and value networks guided Monte Carlo tree search.
38	c	Geometric series $\sum_{i \leq d} b^i = \Theta(b^d)$.
39	b	The Engine could act on anything expressible in its notation, e.g. musical composition.
40	a	“Let us calculate” — reasoning made tangible like mathematics, so disputes resolve by computation.

Next week: search spaces and state space search — depth-first, breadth-first, iterative deepening, and their analysis. Keep the $\langle S, s_0, A, T, G \rangle$ formulation and the $\Theta(b^d)$ growth in mind.